Tema 6

Programación de bases de datos

Base de datos

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 4
1. Introducción, Lenguaje de programación 5
2. Sintaxis. Palabras reservadas. Comentarios 6
2.1. Sintaxis 6
2.2. Palabras reservadas 8
2.3. Comentarios 17
3. Variables del sistema y variables de usuario 22
3.1 Variables del sistema 22
3.2 Variables de usuario 26
4. Tipos de datos compuestos: ENUM, SET 31
4.1 Tipo de datos: ENUM 31
4.2. Tipo de datos: SET 32
4.3. Diferencia entre los tipos de datos ENUM y SET 34
5. Funciones 39
5.1 Introducción a las funciones en MySQL 39
5.2 Funciones predefinidas 39
5.3. Funciones personalizadas 41
5.3.1. La importancia de la sentencia DELIMITER 42
5.3.2 Etiquetado de las funciones personalizadas 43
5.3.3 Ejemplos de funciones personalizadas 45
6. Estructura de control de flujo (disyunción, iteración) 49
6.1. Disyunción: IF 51
6.2. Iteración: WHILE 53
6.3. Iteración: REPEAT 54
7. Procedimientos almacenados 60
7.1. ¿Qué es un procedimiento almacenado? 60
7.2 Características de los procedimientos almacenados 60
7.3 Diferencia entre funciones y procedimientos almacenados 61
7.4 Estructura y ejecución de los procedimientos almacenados 61

7.4.1. Creación de procedimientos almacenados 62
7.4.2. Ejecución de procedimientos almacenados 63
7.5. Ejemplos de procedimientos almacenado 63
7.5.1 Procedimiento almacenado 63
7.5.2 Procedimiento almacenado con lista de parámetros 64
7.5.3 Procedimiento almacenado con lista de parámetros y mensaje de salida 65
7.5.4. Procedimiento almacenado llamado desde otro procedimiento almacenado 67
8. Subrutinas. Ámbito de variables 72
8.1 Subrutinas 72
8.2 Variables globales y locales 72
9. Eventos y disparadores 76
9.1 Eventos 76
9.2 Disparadores 80
10. Excepciones: Tipos y tratamiento 83
10.1. Tipos 83
10.2. Tratamiento 84
11. Cursores 87
12. Resumen 95
Glosario 96
Bibliografía 98

Objetivos de la unidad

Los objetivos de la programación de bases de datos son múltiples y pueden variar según el contexto y la finalidad de las aplicaciones que se estén desarrollando.

Entre los más comunes se encuentran:

- **Almacenamiento y gestión de datos**: Uno de los objetivos más importantes de la programación de bases de datos es el almacenamiento y gestión de grandes cantidades de datos de manera eficiente y organizada. Las bases de datos permiten el almacenamiento estructurado de información y facilitan la manipulación de los datos a través de consultas y otros comandos.
- **Acceso y recuperación de datos**: Proporcionar un acceso rápido y eficiente a los datos almacenados. Los sistemas de gestión de bases de datos (DBMS) ofrecen una variedad de herramientas para acceder y recuperar datos, como consultas SQL, procedimientos almacenados, vistas y otros objetos de la base de datos.
- **Integración de datos**: Integrar datos de diferentes fuentes y sistemas. Esto permite a las empresas y organizaciones trabajar con una sola fuente de datos coherente y compartir información entre diferentes aplicaciones y sistemas.
- **Automatización de procesos**: Los sistemas de bases de datos también Se utilizan para automatizar procesos empresariales y reducir el tiempo y los costos asociados con la gestión de datos.
  Por ejemplo, los procedimientos almacenados y los desencadenadores pueden utilizarse para realizar tareas específicas automáticamente cuando se producen ciertos eventos o condiciones en la base de datos.
- **Análisis de datos**: La programación de bases de datos también puede ayudar en el análisis y la interpretación de grandes cantidades de datos. Las consultas SQL avanzadas y otras herramientas de análisis pueden utilizarse para extraer información significativa de los datos y presentarla de manera fácilmente comprensible para los usuarios.

Algunos de los objetivos enumerados anteriormente ya los hemos ido estudiando en capítulos anteriores. En este, nos centraremos en aquellas cuestiones que nos ayudan a trabajar con las bases de datos de manera optimizada y organizada, pudiendo desarrollar porciones de código que nos faciliten tanto su explotación como su mantenimiento.

1. Introducción, Lenguaje de programación

La programación de bases de datos es una disciplina que involucra la creación de aplicaciones que interactúan con bases de datos para almacenar, recuperar, actualizar y eliminar datos, siendo una parte fundamental del desarrollo de software, ya que la mayoría de las aplicaciones requieren acceso a una base de datos para realizar su trabajo.

Su programación implica el uso de lenguajes de programación y tecnologías específicas para interactuar con una base de datos. Algunos de los lenguajes de programación más comunes utilizados en la programación de bases de datos incluyen **SQL** (**Structured Query Language**), **Java**, **Python** y **PHP**, entre otros.

Las tecnologías utilizadas en la programación de bases de datos incluyen sistemas de gestión de bases de datos (DBMS, por sus siglas en inglés) como **MySQL**, **PostgreSQL**, **Oracle** y **Microsoft SQL Server**, etc., proporcionando herramientas para crear, manipular y administrar bases de datos.

Combinándose con la creación y administración de bases de datos, su programación también incluye la creación de procedimientos almacenados, desencadenadores, vistas y otros objetos de la base de datos que permiten la interacción más avanzada y eficiente con los datos.

2. Sintaxis. Palabras reservadas. Comentarios

2.1. Sintaxis

Dependiendo de cada instrucción, cláusula, o palabra específica, su sintaxis constructiva variará, por lo que la sintaxis específica de cada palabra que estudiaremos en este capítulo, la iremos descubriendo paso a paso.

A modo de recordatorio, visualicemos de nuevo la sintaxis básica de algunas de las instrucciones y cláusulas que hemos visto hasta ahora:

SELECT: Se utiliza para seleccionar datos de una o más tablas.

FROM: Se utiliza para especificar la tabla o tablas de las que se desea seleccionar datos.

WHERE: Se utiliza para especificar las condiciones que deben cumplir las filas seleccionadas.

ORDER BY: Se utiliza para ordenar los resultados de una consulta.

```
SELECT columna1, columna2, ...
FROM tabla
WHERE condicion
ORDER BY columna2;
```

GROUP BY: Se utiliza para agrupar filas que tienen el mismo valor en una o más columnas.

HAVING: Se utiliza para filtrar los resultados de una consulta agrupada.

```
SELECT columna1, SUM(columna2)
FROM tabla
GROUP BY columna1
HAVING SUM(columna2) > 10;
```

JOIN: Se utiliza para combinar datos de dos o más tablas relacionadas.

```
SELECT columna1, columna2, ...
FROM tabla1
JOIN tabla2 ON tabla1.columna = tabla2.columna;
```

INSERT: Se utiliza para insertar nuevos datos en una tabla.

```
INSERT INTO tabla (columna1, columna2, ...)
VALUES (valor1, valor2, ...);
```

UPDATE: Se utiliza para actualizar datos existentes en una tabla.

```
UPDATE tabla
SET columna1 = valor1, columna2 = valor2, ...
WHERE condicion;
```

DELETE: Se utiliza para eliminar filas de una tabla.

```
DELETE FROM tabla
WHERE condicion;
```

CREATE: Se utiliza para crear una nueva tabla, vista o procedimiento almacenado.

```
CREATE TABLE tabla (
   columna1 tipo_dato,
   columna2 tipo_dato,
   ...
);
```

ALTER: Se utiliza para modificar la estructura de una tabla existente.

```
ALTER TABLE tabla
ADD columna tipo_dato;
```

DROP: Se utiliza para eliminar una tabla, vista o procedimiento almacenado.

```
DROP TABLE tabla;
```

2.2. Palabras reservadas

En la tabla que mostramos a continuación, están relacionadas la mayoría de las palabras reservadas de MySQL que, combinadas adecuadamente entre sí, dan lugar a crear consultas, tablas, vistas y procedimientos, manipular datos, y programar porciones de código que nos ayudan a explotar al máximo todas sus posibilidades:

Palabra Reservada	Descripción
ADD	Se usa para agregar una columna a una tabla existente.
ALL	Se usa para especificar que se deben devolver todas las filas, incluidas las duplicadas.
ALTER	Se usa para modificar una tabla existente, como cambiar el nombre de una columna o agregar una restricción.
ANALYZE	Se usa para actualizar las estadísticas de una tabla, lo que puede mejorar el rendimiento de las consultas.
AND	Se usa para combinar dos o más condiciones en una consulta.
AS	Se usa para asignar un alias a una tabla o columna en una consulta.
ASC	Se usa con ORDER BY para ordenar los resultados en orden ascendente.
ASENSITIVE	Se usa para indicar que una consulta debe ser insensible a mayúsculas y minúsculas.
BEFORE	Se usa en disparadores (triggers) para indicar cuándo se debe ejecutar el trigger.
BETWEEN	Se usa para seleccionar valores dentro de un rango específico.
BIGINT	Se usa para definir una columna que almacenará números enteros grandes.
BINARY	Se usa para indicar que una columna debe ser tratada como una cadena de caracteres binarios.
BLOB	Se usa para definir una columna que almacenará datos binarios grandes, como imágenes o archivos.
BOTH	Se usa en una consulta para especificar que ambas condiciones deben ser verdaderas.
BY	Se usa en una consulta GROUP BY para especificar cómo se deben agrupar los resultados.

CALL	Se usa para ejecutar una rutina almacenada (stored procedure) o una función definida por el usuario.
CASCADE	Se usa para especificar que se deben aplicar cambios en cascada a una restricción.
CASE	Se usa para especificar una serie de condiciones y resultados posibles en una consulta.
CHANGE	Se usa para cambiar el nombre de una columna o cambiar su tipo de datos en una tabla existente.
CHAR	Se usa para definir una columna que almacenará caracteres.
CHARACTER	Es sinónimo de CHAR.
CHECK	Se usa para especificar una restricción que debe cumplirse para los valores en una columna.
COLLATE	Se usa para especificar una ordenación de caracteres para una columna o una tabla.
COLUMN	Se usa para especificar una columna en una consulta o para modificar una columna existente en una tabla.
CONDITION	Se usa en declaraciones IF para especificar una condición.
CONSTRAINT	Se usa para especificar una restricción en una tabla, como una clave primaria o externa.
CONTINUE	Se usa en bucles para saltar a la siguiente iteración sin completar la iteración actual.
CONVERT	Se usa para convertir un valor de un tipo de datos a otro.
CREATE	Se usa para crear una nueva tabla, vista o índice.
CROSS	Se usa para combinar datos de dos o más tablas sin una condición común.
CURRENT_DATE	Se usa para devolver la fecha actual.
CURRENT_TIME	Se usa para devolver la hora actual.
CURRENT_TIMESTAMP	Se usa para devolver la fecha y hora actuales.
CURRENT_USER	Se usa para devolver el nombre de usuario actual.
CURSOR	Una declaración que define un cursor, que es un objeto que permite el recorrido de los resultados de una consulta SQL de manera secuencial.

DATABASE	Un identificador para una base de datos en MySQL.
DATABASES	Una palabra clave utilizada en la sentencia SHOW para listar todas las bases de datos en el servidor MySQL.
DAY_HOUR	Una unidad de tiempo que se utiliza para almacenar valores de fecha y hora en MySQL, que representa días y horas.
DAY_MICROSECOND	Una unidad de tiempo que se utiliza para almacenar valores de fecha y hora en MySQL, que representa días y microsegundos.
DAY_MINUTE	Una unidad de tiempo que se utiliza para almacenar valores de fecha y hora en MySQL, que representa días y minutos.
DAY_SECOND	Una unidad de tiempo que se utiliza para almacenar valores de fecha y hora en MySQL, que representa días y segundos.
DEC	Un sinónimo de la palabra clave DECIMAL, que se utiliza para definir un tipo de datos numéricos de coma fija.
DECIMAL	Una palabra clave utilizada para definir un tipo de datos numéricos de coma fija en MySQL.
DECLARE	Una palabra clave utilizada para declarar una variable en una función o procedimiento almacenados.
DEFAULT	Una palabra clave utilizada para definir un valor predeterminado para una columna en una tabla MySQL.
DELAYED	Una palabra clave utilizada con la sentencia INSERT en MySQL, que indica que la inserción debe ser retrasada hasta que no haya otras conexiones leyendo desde la tabla.
DELETE	Una palabra clave utilizada para eliminar filas de una tabla MySQL.
DESC	Una palabra clave utilizada para especificar el orden descendente en la cláusula ORDER BY de una consulta SELECT en MySQL.
DESCRIBE	Una palabra clave utilizada para obtener información sobre la estructura de una tabla MySQL.
DETERMINISTIC	Una palabra clave utilizada para indicar que una función almacenada siempre devuelve el mismo resultado para el mismo conjunto de parámetros.
DISTINCT	Una palabra clave utilizada para seleccionar solo valores únicos en una consulta SELECT en MySQL.

DISTINCTROW	Una palabra clave utilizada en una consulta SELECT en MySQL para seleccionar solo las filas únicas de una tabla.
DIV	Una operación matemática utilizada en MySQL para dividir dos números enteros y devolver un resultado entero.
DOUBLE	Una palabra clave utilizada para definir un tipo de datos numéricos de coma flotante de doble precisión en MySQL.
DROP	Una palabra clave utilizada para eliminar una base de datos, tabla, índice, vista, función o procedimiento almacenados en MySQL.
DUAL	Una tabla especial en MySQL que Se utiliza como un valor por defecto para funciones que no tienen otra tabla de referencia.
EACH	Una palabra clave utilizada para especificar que una consulta SELECT debe procesar las filas de una tabla de forma individual.
ELSE	Una palabra clave utilizada en una instrucción IF en MySQL para especificar una acción a tomar en caso de que la condición IF sea falsa.
ELSEIF	Una palabra clave utilizada en una instrucción IF en MySQL para especificar una condición adicional que se evalúa si la condición IF es falsa.
ENCLOSED	Una palabra clave utilizada para especificar el carácter de delimitación para valores de cadena en una instrucción LOAD DATA INFILE en MySQL.
ESCAPED	Utilizada en la cláusula LIKE para indicar que el siguiente carácter es un carácter de escape.
EXISTS	Utilizada en la cláusula WHERE para verificar la existencia de registros en una subconsulta.
EXIT	Utilizada en la declaración de un bloque de código para salir de él antes de que finalice.
EXPLAIN	Utilizada para obtener información sobre cómo MySQL está ejecutando una consulta.
FALSE	Valor booleano falso.
FETCH	Utilizada para recuperar una fila de un cursor.
FLOAT	Tipo de dato para números de coma flotante de precisión simple.
FLOAT4	Tipo de dato para números de coma flotante de precisión simple.
FLOAT8	Tipo de dato para números de coma flotante de doble precisión.

FOR	Utilizada en la declaración de un cursor para establecer una iteración.
FORCE	Utilizada para forzar el uso de un índice específico en una consulta.
FOREIGN	Utilizada en la creación de una tabla para establecer una clave externa.
FROM	Utilizada en la cláusula SELECT para especificar las tablas de las que se obtendrán los datos.
FULLTEXT	Utilizada en la creación de una tabla para crear un índice de texto completo en una columna.
GOTO	Utilizada para saltar a una etiqueta específica en el código.
GRANT	Utilizada para conceder privilegios a un usuario en una base de datos.
GROUP	Utilizada en la cláusula SELECT para agrupar filas que tengan valores comunes en una columna.
HAVING	Utilizada en la cláusula SELECT para filtrar grupos que no cumplan una condición.
HIGH_PRIORITY	Utilizada en una consulta INSERT para indicar que la consulta tiene alta prioridad.
HOUR_MICROSECOND	Tipo de dato para un intervalo de tiempo con microsegundos y horas.
HOUR_MINUTE	Tipo de dato para un intervalo de tiempo con minutos y horas.
HOUR_SECOND	Tipo de dato para un intervalo de tiempo con segundos y horas.
IF	Utilizada en la creación de una función para ejecutar un bloque de código si se cumple una condición.
IGNORE	Utilizada en una consulta INSERT para ignorar los errores que se produzcan al insertar registros.
IN	Utilizada en la cláusula WHERE para especificar un conjunto de valores.
INDEX	Utilizada en la creación de una tabla para crear un índice en una o más columnas.
INFILE	Utilizada en una consulta LOAD DATA para especificar el archivo que se cargará en la tabla.
INNER	Utilizada en la cláusula JOIN para especificar que se deben devolver solo las filas que tengan coincidencias en ambas tablas.
INOUT	Utilizada en la creación de una función para especificar que un parámetro Se utilizará para pasar valores a la función y para devolver valores desde la función.

INSENSITIVE	Utilizada en la cláusula SELECT para indicar que la comparación de cadenas debe ser insensible a mayúsculas y minúsculas.
INSERT	Utilizada para insertar nuevos registros en una tabla.
INT	Tipo de dato para números enteros.
INT1	Tipo de dato para números enteros de 1 byte.
INT2	Tipo de dato para números enteros de 2 bytes.
INT3	Un alias para el tipo de datos INT, representa un entero con longitud de 3 bytes.
INT4	Un alias para el tipo de datos INT, representa un entero con longitud de 4 bytes.
INT8	Un alias para el tipo de datos BIGINT, representa un entero con longitud de 8 bytes.
INTEGER	Un sinónimo para el tipo de datos INT.
INTERVAL	Se utiliza para definir un intervalo de tiempo, se puede utilizar en una cláusula WHERE, ORDER BY o GROUP BY.
INTO	Se utiliza para especificar la tabla en la que se insertarán los datos que se obtienen de una consulta SELECT.
IO_AFTER_GTIDS	Se utiliza en una declaración de evento para especificar el momento en que se debe ejecutar un evento.
IO_BEFORE_GTIDS	Se utiliza en una declaración de evento para especificar el momento en que se debe ejecutar un evento.
IS	Se utiliza en una cláusula WHERE para comparar dos valores. También se utiliza en combinación con NULL para comprobar si un valor es NULL o no.
ITERATE	Se utiliza en una declaración WHILE o LOOP para volver a la siguiente iteración del bucle.
JOIN	Se utiliza en una cláusula SELECT para combinar filas de dos o más tablas en función de una columna relacionada entre ellas.
KEY	Se utiliza para especificar un índice en una tabla.
KEYS	Un sinónimo de la palabra clave KEY.

KILL	Se utiliza para cancelar una consulta que se está ejecutando actualmente.
LEADING	Se utiliza en una cláusula INDEX para indicar que MySQL debe usar el índice en la columna más a la izquierda de la lista de columnas del índice.
LEAVE	Se utiliza en una declaración WHILE o LOOP para salir del bucle.
LEFT	Se utiliza en una cláusula JOIN para indicar que se deben incluir todas las filas de la tabla izquierda y solo las filas coincidentes de la tabla derecha.
LIKE	Se utiliza en una cláusula WHERE para buscar patrones en una columna.
LIMIT	Se utiliza para limitar el número de filas que seDevuelven en una consulta SELECT.
LINEAR	Se utiliza en una cláusula INDEX para especificar que MySQL debe utilizar un índice de búsqueda lineal en lugar de un índice basado en árbol B.
LINES	Se utiliza en una cláusula LOAD DATA INFILE para especificar cómo se deben interpretar los saltos de línea en el archivo.
LOAD	Se utiliza para cargar datos en una tabla desde un archivo CSV, TSV o SQL.
LOCALTIME	Devuelve la hora actual en la zona horaria del sistema.
LOCALTIMESTAMP	Devuelve la fecha y hora actual en la zona horaria del sistema.
LOCK	Se utiliza para bloquear una tabla o una fila específica en una tabla.
LONG	Un sinónimo del tipo de datos MEDIUMTEXT.
LONGBLOB	Un tipo de datos que almacena grandes cantidades de datos binarios.
LONGTEXT	Un tipo de datos que almacena grandes cantidades de datos de texto.
LOOP	Se utiliza para crear un bucle repetitivo en una declaración almacenada.
LOW_PRIORITY	Se utiliza en una consulta INSERT o UPDATE para dar prioridad a las solicitudes de lectura frente a las de escritura.
MASTER_BIND	Esta opción Se usa con las declaraciones SELECT para especificar en qué servidor esclavo se ejecutará la consulta.

MASTER_SSL_VERIFY_SERVER_CERT	Esta opción Se usa para especificar si se debe verificar el certificado del servidor maestro para una conexión SSL.
MATCH	Esta palabra reservada Se utiliza en la cláusula WHERE para especificar cómo se deben comparar las palabras en una búsqueda de texto completo.
MAXVALUE	Esta palabra reservada Se utiliza en la definición de una secuencia y Se utiliza para especificar el valor máximo que puede tomar la secuencia.
MEDIUMBLOB	Es un tipo de datos que Se utiliza para almacenar grandes objetos binarios en MySQL. Este tipo de dato puede almacenar hasta 16 MB de datos binarios.
MEDIUMINT	Es un tipo de dato entero que puede almacenar valores enteros en el rango de -8388608 a 8388607.
MEDIUMTEXT	Es un tipo de datos de texto que Se utiliza para almacenar grandes cantidades de texto en MySQL. Este tipo de dato puede almacenar hasta 16 MB de datos de texto.
MIDDLEINT	Es un tipo de dato entero que puede almacenar valores enteros en el rango de -8388608 a 8388607.
MINUTE_MICROSECOND	Es un tipo de intervalo de tiempo que representa minutos y microsegundos.
MINUTE_SECOND	Es un tipo de intervalo de tiempo que representa minutos y segundos.
MOD	Esta función Se utiliza para obtener el resto de la división de dos números.
MODIFIES	Esta palabra reservada Se utiliza en la definición de procedimientos almacenados y Se utiliza para indicar que el procedimiento modifica los datos de la base de datos.
NATURAL	Esta palabra reservada Se utiliza en la cláusula JOIN para indicar que se deben unir las tablas en función de las columnas con los mismos nombres.
NOT	Esta palabra reservada Se utiliza para negar una condición en una expresión booleana.
NO_WRITE_TO_BINLOG	Esta opción Se utiliza para indicar que una instrucción no debe escribirse en el registro binario.
NULL	Es un valor especial que Se utiliza para indicar que un campo no tiene un valor asignado.
NUMERIC	Es un tipo de datos que Se utiliza para almacenar números de alta precisión.
ON	Esta palabra reservada Se utiliza en la cláusula JOIN para especificar la condición de unión.
OPTIMIZE	Esta palabra reservada Se utiliza para optimizar una tabla.

OPTION	Esta palabra reservada Se utiliza en la cláusula ORDER BY para especificar una opción de clasificación.
OPTIONALLY	Esta palabra reservada Se utiliza en la cláusula INSERT para indicar que los valores de algunas columnas pueden ser nulos.
OR	Esta palabra reservada Se utiliza para especificar múltiples condiciones en una expresión booleana.
ORDER	Esta palabra reservada Se utiliza en la cláusula SELECT para especificar el orden en el que se deben devolver los resultados.
OUT	Esta palabra reservada Se utiliza en la definición de procedimientos almacenados para especificar un parámetro de salida.
OUTER	Esta palabra reservada Se utiliza en la cláusula JOIN para especificar unir las tablas en función de una condición determinada, incluyendo filas que no cumplen la condición.
OUTFILE	Esta palabra reservada Se utiliza para escribir los resultados de una consulta en un archivo.
PARTITION	Se utiliza para definir cómo se van a dividir los datos en particiones para una tabla particionada. Con PARTITION se pueden mejorar el rendimiento y la administración de tablas grandes, ya que se pueden realizar operaciones específicas en cada partición.
PRECISION	Se utiliza para especificar el número máximo de dígitos que se pueden mostrar en un tipo de datos DECIMAL o NUMERIC. En el caso de FLOAT y DOUBLE, la precisión se refiere al número de dígitos significativos.
PRIMARY	Se utiliza para especificar la clave primaria de una tabla. La clave primaria es una columna o un conjunto de columnas que identifican de manera única cada fila en una tabla.
PROCEDURE	Se utiliza para definir una rutina que se ejecuta en la base de datos. Las rutinas almacenadas, incluidos los procedimientos almacenados, son un conjunto de sentencias SQL que se pueden reutilizar para realizar una tarea específica en la base de datos.
PURGE	Se utiliza para borrar permanentemente los binlogs que ya no son necesarios. Los binlogs son archivos de registro de transacciones que Se utilizan en la replicación de bases de datos MySQL.
RANGE	Se utiliza para definir cómo se van a dividir los datos en rangos para una tabla particionada. Con RANGE se pueden mejorar el rendimiento y la administración

	de tablas grandes, ya que se pueden realizar operaciones específicas en cada rango.
READ	Se utiliza para especificar el tipo de bloqueo de lectura que Se utilizará en una transacción o en una consulta. Puede ser READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ o SERIALIZABLE.
READS	Se utiliza para especificar si se deben bloquear las tablas en lectura o en escritura en una transacción.
READ_WRITE	Se utiliza para especificar que una tabla debe ser bloqueada tanto para lectura como para escritura. Es el comportamiento predeterminado en MySQL.

Excedería las dimensiones de este manual poner un ejemplo de cada una de ellas, pero a lo largo de los apartados de este capítulo iremos viendo algunos ejemplos de distintas combinaciones de palabras reservadas.

2.3. Comentarios

En MySQL, hay dos formas de agregar comentarios:

- **Comentarios de una línea**
  Utilizados para agregar comentarios en una sola línea. Se pueden utilizar dos tipos de caracteres:
  - Utilizando los dos guiones (**--**) en el inicio de la línea. Todo lo que se escriba después de los dos guiones se considerará un comentario.
  - Utilizando el símbolo de almohadilla (**#**) en el inicio de la línea. Todo lo que se escriba después del símbolo de almohadilla se considerará un comentario.

- **Comentarios de varias líneas**
  Se usan para agregar comentarios que abarcan varias líneas. Se pueden utilizar dos tipos de caracteres para agregar comentarios de varias líneas:
  - Utilizando **/*** para abrir el comentario y ***/** para cerrarlo. Todo lo que se escriba entre estos dos caracteres se considerará un comentario.
  - Utilizando **--** al final de cada línea del comentario. MySQL ignorará todo lo que se escriba después de **--** hasta el final de la línea. La sintaxis de multi-línea en MySQL es /* … */

Ejemplo de comentarios:

```
#Esto es un comentario
-- Esto es otro comentario
/*
Y esto otro comentario
*/
SELECT *
FROM Music; -- Y esto también es un comentario
```

Importante

Los comentarios no afectan la ejecución de la consulta y son útiles para documentar la lógica del código o para agregar notas importantes para otros usuarios que puedan leer la consulta en el futuro.

Veamos de forma práctica cómo se lleva a cabo la gestión de asistencias en FitLife (uso de palabras reservadas)

El gimnasio FitLife registra cada vez que un socio asiste a una clase en la tabla Asistencia. Además, existe la tabla Miembros. Vamos a usar diversas palabras reservadas de MySQL para crear, poblar y consultar estos datos.

Tareas a realizar:

1. **Crear la tabla**

Define la tabla Asistencia con las columnas:

- id (INT, clave primaria auto incremental)
- miembro_id (INT, clave foránea a Miembros(id))
- clase (VARCHAR(50))
- fecha (DATE)

2. **Insertar datos**

Inserta cinco registros de asistencia usando INSERT.

3. **Listar clases únicas**

Muestra todas las clases distintas a las que se ha asistido (SELECT DISTINCT).

4. **Contar asistencias por clase**

Obtén la lista de clases junto con el número de asistencias, ordenada de mayor a menor (GROUP BY, HAVING, ORDER BY).

5. **Corregir una asistencia**

Supón que el registro con id = 3 debería ser de la clase "Spinning" en lugar de "Yoga". Haz el cambio con UPDATE.

6. **Eliminar asistencias antiguas**

Borra todos los registros anteriores a 2025-01-01 con DELETE.

7. **Ampliar la tabla**

Agrega una columna duracion (INT minutos) a Asistencia con ALTER TABLE.

Soluciones

1. Crear la tabla

```
CREATE TABLE Asistencia (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  miembro_id   INT,
  clase       VARCHAR(50),
  fecha       DATE,
  FOREIGN KEY (miembro_id) REFERENCES Miembros(id)
);
```

2. Insertar datos

```
INSERT INTO Asistencia (miembro_id, clase, fecha) VALUES
  (1, 'Yoga',      '2025-07-10'),
  (2, 'Pilates',   '2025-07-11'),
  (1, 'Yoga',      '2025-07-12'),
```

```
(3, 'Spinning',   '2025-07-12'),
(2, 'CrossFit',   '2025-07-13');
```

3. Listar clases únicas

```
SELECT DISTINCT clase
FROM Asistencia;
```

4. Contar asistencias por clase

```
SELECT clase, COUNT(*) AS total_asistencias
FROM Asistencia
GROUP BY clase
HAVING COUNT(*) > 1
ORDER BY total_asistencias DESC;
```

5. Corregir una asistencia (id = 3 cambia 'Yoga' → 'Spinning')

```
UPDATE Asistencia
SET clase = 'Spinning'
WHERE id = 3;
```

6. Eliminar asistencias anteriores a 2025-01-01

```
DELETE FROM Asistencia
WHERE fecha < '2025-01-01';
```

7. Ampliar la tabla para incluir duración de la clase

```
ALTER TABLE Asistencia
ADD COLUMN duracion INT;
```

Palabras reservadas destacadas

- CREATE, ALTER, DROP
- INSERT, VALUES
- SELECT, DISTINCT, FROM, WHERE
- GROUP BY, HAVING, ORDER BY
- UPDATE, SET
- DELETE
- PRIMARY KEY, FOREIGN KEY

Tech Hub <FP> euroformac

3. Variables del sistema y variables de usuario

3.1 Variables del sistema

En **MySQL**, las variables del sistema son parámetros configurables que controlan el comportamiento y el rendimiento del servidor de base de datos.

Estas variables pueden ser ajustadas por el administrador del sistema para mejorar el rendimiento y la estabilidad del servidor, así como para personalizar el comportamiento del servidor para satisfacer las necesidades específicas del entorno de la aplicación.

Algunos de los objetivos principales de las variables del sistema en **MySQL** son:

- **Controlar el uso de recursos**: Las variables del sistema pueden ser utilizadas para controlar el uso de recursos del servidor, como la memoria y la CPU, para optimizar el rendimiento y evitar cuellos de botella.

- **Mejorar la seguridad**: Algunas variables del sistema pueden ser utilizadas para mejorar la seguridad del servidor, como la limitación de conexiones concurrentes o la configuración de contraseñas seguras.

- **Personalizar el comportamiento del servidor**: También pueden ser utilizadas para personalizar el comportamiento del servidor y adaptarlo a las necesidades específicas de la aplicación. Por ejemplo, se pueden ajustar las variables para optimizar el rendimiento de las consultas o para habilitar o deshabilitar características específicas del servidor.

A continuación, se presentan algunas de las **variables de sistema más comunes en MySQL**, aunque la lista es mucho más extensa:

Variable del sistema	Descripción
innodb_buffer_pool_size	La cantidad de memoria que se asigna a la memoria intermedia de InnoDB, que es el espacio de almacenamiento temporal que utiliza InnoDB para las tablas y los índices.
innodb_file_per_table	Esta variable determina si InnoDB crea una tabla en su propio archivo

	de datos o comparte un archivo con varias tablas.
innodb_flush_log_at_trx_commit	Controla el comportamiento de escritura de registros de InnoDB y el compromiso de transacciones.
innodb_log_file_size	El tamaño de los archivos de registro de transacciones de InnoDB.
innodb_log_buffer_size	El tamaño del búfer de registro de transacciones de InnoDB.
innodb_thread_concurrency	El número máximo de subprocesos que InnoDB puede utilizar simultáneamente.
innodb_flush_method	Controla el método de escritura de InnoDB.
innodb_io_capacity	El número máximo de solicitudes de E/S que InnoDB puede procesar simultáneamente.
innodb_read_io_threads	El número de subprocesos que InnoDB puede usar para leer datos de disco.
innodb_write_io_threads	El número de subprocesos que InnoDB puede usar para escribir datos en disco.
key_buffer_size	La cantidad de memoria que se asigna para el caché de índices en memoria.
log_bin	Esta variable habilita la funcionalidad de registro binario, lo que significa que las declaraciones SQL se registran en un archivo para la replicación de bases de datos.
log_slow_queries	Esta variable habilita el registro de consultas lentas, lo que ayuda a los administradores de bases de datos a identificar los cuellos de botella del rendimiento.
max_connections	El número máximo de conexiones de clientes que se permiten al mismo tiempo en el servidor MySQL.
query_cache_size	Esta variable define el tamaño del caché de consultas, que almacena los resultados de las consultas anteriores para que se puedan recuperar más rápido en el futuro. Si se configura correctamente, esta

	variable puede mejorar significativamente el rendimiento de la base de datos.
slow_query_log_file	La ubicación del archivo donde se almacenan los registros de consultas lentas.
table_open_cache	El número de tablas que se pueden abrir simultáneamente.
thread_cache_size	El número máximo de subprocesos que se pueden almacenar en caché en el servidor.
tmp_table_size	Esta variable define el tamaño máximo de las tablas temporales utilizadas por MySQL. El valor predeterminado es pequeño, por lo que si se están utilizando consultas complejas que requieren la creación de tablas temporales, es posible que deba aumentar esta variable para evitar errores.
wait_timeout	Esta variable define el tiempo máximo que una conexión puede estar inactiva antes de que MySQL la cierre automáticamente. Esto es importante para limitar la cantidad de conexiones inactivas y liberar recursos de memoria.

Estas son solo algunas de las variables de sistema más comunes en MySQL. Para ver la lista completa de variables de sistema, puede ejecutar el comando **SHOW VARIABLES** en una sesión de MySQL.

Limit to 1000 rows

```
1 •  SHOW VARIABLES;
2
3
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Variable_name	Value
activate_all_roles_on_login	OFF
admin_address	
admin_port	33062
admin_ssl_ca	
admin_ssl_capath	
admin_ssl_cert	
admin_ssl_cipher	
admin_ssl_crl	
admin_ssl_crlpath	
admin_ssl_key	
admin_tls_ciphersuites	
admin_tls_version	TLSv1.2,TLSv1.3
authentication_policy	*,,
auto_generate_certs	ON
auto_increment_increment	1
auto_increment_offset	1
autocommit	ON
automatic_sp_privileges	ON
avoid_temporal_upgrade	OFF
back_log	151
basedir	C:\Program Files\MySQL\MySQL Server 8.0\
big_tables	OFF
bind_address	*
binlog_cache_size	32768
binlog_checksum	CRC32
binlog_direct_non_transac...	OFF
binlog_encryption	OFF
binlog_error_action	ABORT_SERVER
binlog_expire_logs_auto_...	ON
binlog_expire_logs_seconds	2592000
binlog_format	ROW

Lista de variables del sistema en una sesión MySQL

3.2 Variables de usuario

En MySQL, además de las variables del sistema, también existen las variables de usuario, que son variables que pueden ser definidas y utilizadas por los usuarios para almacenar información temporal en el contexto de una sesión de conexión específica.

Las variables de usuario en MySQL se pueden definir utilizando la sintaxis **@nombre_variable** y se pueden utilizar en cualquier momento dentro de la sesión de conexión actual.

Estas variables se pueden utilizar para almacenar valores que se necesitan en varias consultas dentro de una sesión, o para realizar cálculos o transformaciones de datos en el servidor.

Importante

MySQL no tiene variables de usuario tipadas. Todas las variables de usuario en MySQL son tratadas como cadenas de caracteres (strings) y no están asociadas a un tipo de datos específico.

Sin embargo, se pueden realizar conversiones de tipo de datos utilizando las funciones de conversión de MySQL, como **CAST** o **CONVERT**.

Ejemplo de conversión de una Variable de usuario al tipo de dato deseado:

Por ejemplo, para convertir una variable de usuario llamada **mi_variable** en un entero, se puede utilizar la siguiente sintaxis:

```
SELECT CAST(@mi_variable AS INT);
```

En este caso, la función **CAST** convierte el valor de la variable **mi_variable** en un entero.

Importante

Debido a que las variables de usuario en MySQL no están asociadas a un tipo de datos específico, es responsabilidad del/a programador/a asegurarse de que los valores almacenados en estas variables sean compatibles con el tipo de datos que se espera en las consultas y operaciones en las que se utilizan. En caso contrario, se pueden producir errores de conversión de tipos o resultados inesperados.

Siguiendo con la cuestión del tipado de datos, aunque todas las variables de usuario son cadenas de caracteres, es posible definir variables que actúen como contadores o que se comporten como variables booleanas.

Ejemplo de variable de usuario utilizada como contador:

Por ejemplo, para definir una variable de tipo contador llamada **@contador,** que se inicializa en cero y se incrementa en uno por cada registro de una tabla, se puede utilizar la siguiente sintaxis:

SET @contador = 0;

SELECT *, @contador**:=**@contador+1 AS NumeroDeFila FROM mi_tabla;ç

En este ejemplo, la variable **@contador** se inicializa en cero y se incrementa en uno por cada registro seleccionado de la tabla **mi_tabla**. La columna calculada **NumeroDeFila** utiliza el valor de la variable **@contador** para numerar cada registro.

Por tanto, aunque todas las variables de usuario en MySQL son de tipo cadena de caracteres, es posible utilizar técnicas y estrategias para simular otros tipos de datos, como contadores o variables booleanas, mediante la manipulación y conversión de las cadenas de caracteres almacenadas en las variables.

Ejemplo de una variable de usuario:

Por ejemplo, para definir una variable de usuario llamada **mi_variable** con un valor de **10**, se puede utilizar la siguiente sintaxis:

```
/*
Más adelante veremos que hay otros dos tipos de variables locales, además del tipo "SET", que se pueden usar en procedimientos almacenados, funciones y desencadenadores (triggers).
*/
SET @mi_variable = 10;
```

Una vez definida la variable, se puede utilizar en cualquier consulta dentro de la sesión actual. Por ejemplo, para utilizar la variable en una consulta SELECT, se puede utilizar la siguiente sintaxis:

```
SELECT * FROM mi_tabla
WHERE mi_columna > @mi_variable;
```

En este caso, la consulta seleccionará todas las filas de la tabla **mi_tabla** donde el valor de la columna mi_columna sea mayor que el valor de la variable **mi_variable**.

Importante

Las variables de usuario en MySQL son específicas de la sesión de conexión actual y no se pueden utilizar en otras sesiones de conexión.

Una vez que se cierra la sesión de conexión, las variables de usuario definidas en esa sesión se eliminan automáticamente.

Siguiendo con nuestro ejemplo práctico, veamos cómo ajustar variables - Importación y análisis de reservas en FitLife

FitLife acaba de recibir un archivo con 1 000 000 de reservas de clase históricas para migrar a la tabla ReservasClases. Queremos:

1. **Ajustar variables del sistema** para optimizar la importación masiva.
2. **Importar** los datos en una única transacción.

3. **Generar un informe** que numere secuencialmente cada reserva por miembro, usando variables de usuario.

1. Ajuste de variables del sistema

Antes de la importación, elevamos varios parámetros de rendimiento (requiere privilegios de administrador y motor InnoDB):

```
-- Permitir tablas temporales más grandes (default pequeño)
SET GLOBAL tmp_table_size      = 256 * 1024 * 1024;  -- 256 MB
SET GLOBAL max_heap_table_size = 256 * 1024 * 1024;  -- 256 MB

-- Aumentar conexiones para procesos concurrentes
SET GLOBAL max_connections     = 300;

-- Reservar más memoria para InnoDB
SET GLOBAL innodb_buffer_pool_size      = 2 * 1024 * 1024 * 1024; -- 2 GB
SET GLOBAL innodb_flush_log_at_trx_commit = 1;  -- seguridad máxima
```

2. Importación masiva dentro de una transacción

Importamos todas las reservas en bloque (suponiendo que ya hemos cargado el CSV en una tabla temporal TempReservas):

```
START TRANSACTION;

INSERT INTO ReservasClases (miembro_id, fecha_clase, nombre_clase)
SELECT miembro_id, fecha_clase, nombre_clase
FROM TempReservas;

COMMIT;
```

START TRANSACTION … COMMIT agrupa la inserción en un solo lote atómico.

Si hubiera error, un ROLLBACK devolvería la tabla a su estado previo.

3. Informe con numeración por miembro (variables de usuario)

Queremos un listado ordenado en el que, para cada miembro_id, las reservas aparezcan con un número de fila num_reserva:

```
-- Inicializamos variables de usuario
SET @prev_member := 0;
SET @counter     := 0;

-- Generamos el informe
SELECT
  miembro_id,
  fecha_clase,
  nombre_clase,
  @counter := IF(@prev_member = miembro_id, @counter + 1, 1) AS num_reserva,
  @prev_member := miembro_id
FROM ReservasClases
ORDER BY miembro_id, fecha_clase;
```

Cómo funciona

- @prev_member guarda el último miembro_id leído.
- @counter se incrementa si es el mismo miembro, o se reinicia a 1 al cambiar de miembro.
- El ORDER BY asegura que las filas vienen agrupadas por miembro_id y ordenadas por fecha_clase.

Con ello, mezclamos variables de sistema (optimización de importación) y variables de usuario (creación de un contador dentro de un SELECT) para completar la tarea de forma eficiente y fiable.

4. Tipos de datos compuestos: ENUM, SET

4.1 Tipo de datos: ENUM

En MySQL, los enumerados son un tipo de datos compuesto que permite definir un conjunto de valores posibles para una columna de una tabla.

Un enumerado se define como una lista separada por comas de los valores posibles entre paréntesis, y se utiliza para restringir los valores que se pueden insertar en una columna.

Ejemplo de ENUMERADO:

Por ejemplo, si se desea definir una columna llamada estado que sólo acepte valores de "activo", "inactivo" o "suspendido", se podría definir la columna con el tipo de datos:

```
ENUM('activo', 'inactivo', 'suspendido')
```

De esta manera, sólo se permitirán valores de la lista definida en la columna.

Importante

El uso de enumerados puede ser limitado en comparación con otros tipos de datos, ya que una vez definido el conjunto de valores posibles, no se pueden agregar o eliminar valores sin modificar la estructura de la tabla.

Por otro lado, el uso excesivo de enumerados puede afectar el rendimiento de las consultas y la escalabilidad del esquema de la base de datos, por lo que es recomendable utilizar los enumerados con moderación y en situaciones donde la lista de valores posibles sea finita y no cambie con frecuencia.

Ejemplo 2 de ENUMERADO:

```
-- Creación de la tabla con una columna ENUM
CREATE TABLE ejemplo_enum (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(50),
  genero ENUM('Masculino', 'Femenino', 'No binario')
```

```
);

-- Inserción de datos en la tabla
INSERT INTO ejemplo_enum (nombre, genero)
VALUES ('Juan', 'Masculino'),
    ('María', 'Femenino'),
    ('Pat', 'No binario'),
    ('Pedro', 'Masculino'),
    ('Ana', 'Femenino');

-- Consulta de la columna ENUM
SELECT nombre, genero
FROM ejemplo_enum
WHERE genero = 'Femenino' OR genero = 'No binario';
```

En este ejemplo, creamos una tabla llamada "ejemplo_enum" con tres columnas: "id", "nombre" y "genero". La columna "id" se define como una clave primaria autoincremental, mientras que la columna "nombre" se define como una cadena de caracteres de hasta 50 caracteres.

La columna "genero" se define como una columna **ENUM** que solo puede contener valores predefinidos de "**Masculino**", "**Femenino**" o "**No binario**".

Luego, insertamos cinco filas en la tabla, cada una con un nombre y un valor de género. Finalmente, consultamos la tabla para obtener los nombres y los valores de género de cada fila.

4.2. Tipo de datos: SET

En **MySQL**, el tipo de datos **SET** es un tipo de datos que permite almacenar uno o varios valores de un conjunto de opciones predefinidas en una columna de una tabla.

Los valores se separan por comas y se almacenan como una cadena de caracteres.

La sintaxis básica para crear una columna SET en MySQL es la siguiente:

column_name SET(`value1´, `value2´, `value3´, ...)

Donde "**column_name**" es el nombre de la columna y **"value1, value2, value3, ..."** son los valores posibles que puede tomar la columna.

Por ejemplo, para crear una tabla con una columna SET llamada "genero" que puede tomar los valores "Masculino", "Femenino" y "No binario", se puede usar la siguiente sintaxis:

Ejemplo de tipo de dato SET:

```
# Creamos una tabla llamada ejemplo que contendrá una columna de tipo SET
CREATE TABLE ejemplo (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(50),
  genero SET('Masculino', 'Femenino', 'No binario')
);
```

Una vez creada la tabla, se pueden insertar valores en la columna SET de la siguiente manera:

```
INSERT INTO ejemplo (nombre, genero) VALUES ('Juan', 'Masculino,Femenino');
```

En este ejemplo, se inserta un valor en la columna "genero" que corresponde a dos opciones: "Masculino" y "Femenino".

Para buscar valores en una columna SET, se puede utilizar la función **FIND_IN_SET()** de MySQL.

SELECT nombre FROM ejemplo WHERE FIND_IN_SET('Femenino', **genero**);

Esta consulta buscará en la columna "genero" de la tabla "ejemplo" los valores "Femenino" y devolverá los nombres correspondientes.

4.3. Diferencia entre los tipos de datos ENUM y SET

Las principales **diferencias entre ENUM y SET** en MySQL son las siguientes:

Valores permitidos	**ENUM** permite almacenar un solo valor de una lista de opciones predefinidas, mientras que **SET** permite almacenar uno o varios valores de una lista de opciones predefinidas.
Representación en la tabla	Ambos tipos se almacenan internamente como valores numéricos (TINYINT/SMALLINT para ENUM, mapa de bits para SET), no como cadenas. Las cadenas son sólo la representación de salida.
Tamaño de almacenamiento	**ENUM** utiliza un byte para almacenar hasta 255 valores posibles, mientras que **SET** utiliza un byte adicional para cada 8 valores posibles.
Consultas	Para buscar valores en una columna **ENUM**, se utiliza el operador **"="** y para buscar valores en una columna **SET**, se utiliza la función **FIND_IN_SET()**.
Flexibilidad	**SET** es más flexible que **ENUM** ya que permite seleccionar varios valores para una misma fila, mientras que **ENUM** solo permite seleccionar un valor.
Significado de los valores	**ENUM** puede ser utilizado para almacenar valores con un significado específico y fijo, mientras que **SET** se utiliza para almacenar valores que no necesariamente tienen un significado específico y pueden ser opcionales.

Importante

ENUM es una buena opción para columnas que contienen valores fijos y definidos, mientras que **SET** es mejor para columnas que contienen valores opcionales y flexibles.

Importante

ENUM es una buena opción para columnas que contienen valores fijos y definidos, mientras que **SET** es mejor para columnas que contienen valores opcionales y flexibles.

Independientemente de la elección entre **ENUM** o **SET**, ambos tipos de datos son útiles para limitar los valores posibles que se pueden almacenar en una columna y ayudan a garantizar la integridad de los datos, aunque se recomienda que se limiten la cantidad de valores posibles en las columnas **SET** y **ENUM** para evitar problemas de rendimiento, ya que a medida que la cantidad de valores posibles aumenta, el rendimiento de la consulta puede verse afectado, independientemente del tipo de datos utilizado.

A continuación, veamos cómo aplicar ENUM y SET - Gestión de miembros y sus preferencias en FitLife (ENUM y SET)

FitLife quiere llevar un registro de sus socios, su estado de alta y las actividades que les interesan. Para ello, se utilizarán:

Un campo estado de tipo ENUM, con valores posibles: 'activo', 'inactivo', 'suspendido'.

Un campo intereses de tipo SET, con valores posibles: 'yoga', 'spinning', 'crossfit', 'pilates'.

Tareas a realizar

Crear la tabla

Define la tabla Miembros con:

id (INT auto incremental, clave primaria)

nombre (VARCHAR(50))

estado (ENUM('activo','inactivo','suspendido'), por defecto 'activo')

intereses (SET('yoga','spinning','crossfit','pilates'), por defecto vacío)

Insertar miembros

a. Dar de alta a “Laura Gómez” con estado por defecto y sin intereses aún.
b. Dar de alta a “Carlos Díaz” con estado 'activo' e intereses en 'yoga' y 'pilates'.

c. Intentar insertar a “Marina Ruiz” con estado 'suspendido' e intereses 'boxeo' (no permitido) y observar el error.

Consultar socios activos

Muestra id, nombre y estado de todos los miembros cuyo estado = 'activo'.

Buscar interesados en spinning

Listar nombre de los miembros que tengan 'spinning' en su conjunto de intereses.

Actualizar intereses

Para “Laura Gómez” añade también intereses en 'crossfit' y 'spinning'.

Cambiar estado

Pasa a “Carlos Díaz” de 'activo' a 'suspendido'.

Ampliar ENUM

Imagina que añades el estado 'visitante'. Muestra la sentencia ALTER TABLE necesaria para incluirlo en el ENUM.

Soluciones

```
-- 1. Crear la tabla Miembros
CREATE TABLE Miembros (
  id        INT AUTO_INCREMENT PRIMARY KEY,
  nombre    VARCHAR(50),
  estado    ENUM('activo','inactivo','suspendido') DEFAULT 'activo',
  intereses SET('yoga','spinning','crossfit','pilates') DEFAULT ''
);

-- 2a. Insertar a Laura con valores por defecto
INSERT INTO Miembros (nombre) VALUES ('Laura Gómez');

-- 2b. Insertar a Carlos con intereses múltiples
INSERT INTO Miembros (nombre, intereses)
VALUES ('Carlos Díaz', 'yoga,pilates');
```

```
-- 2c. Intento de inserción inválido (estado OK, interés 'boxeo' no existe)
INSERT INTO Miembros (nombre, estado, intereses)
VALUES ('Marina Ruiz', 'suspendido', 'boxeo');
-- ERROR: “Incorrect SET value: 'boxeo' for column 'intereses'”

-- 3. Consultar miembros activos
SELECT id, nombre, estado
FROM Miembros
WHERE estado = 'activo';

-- 4. Buscar socios con spinning en intereses
SELECT nombre
FROM Miembros
WHERE FIND_IN_SET('spinning', intereses);

-- 5. Añadir 'crossfit' y 'spinning' a Laura (id=1)
UPDATE Miembros
SET intereses = CONCAT_WS(',', intereses, 'crossfit', 'spinning')
WHERE nombre = 'Laura Gómez';

-- 6. Cambiar estado de Carlos a 'suspendido'
UPDATE Miembros
SET estado = 'suspendido'
WHERE nombre = 'Carlos Díaz';

-- 7. Ampliar el ENUM para incluir 'visitante'
ALTER TABLE Miembros
MODIFY COLUMN estado
  ENUM('activo','inactivo','suspendido','visitante')
```

DEFAULT 'activo';

Explicación de lo aprendido

ENUM restringe el campo estado a un conjunto finito de valores, con validación en inserción/actualización.

SET permite múltiples elecciones combinadas en intereses, y se consulta con FIND_IN_SET().

Control de errores al intentar valores no permitidos.

Uso de ALTER TABLE … MODIFY COLUMN para ampliar el ENUM.

5. Funciones

5.1 Introducción a las funciones en MySQL

Las funciones en MySQL son bloques de código reutilizables que se utilizan para realizar una tarea específica y devolver un valor como resultado. Estas funciones pueden ser predefinidas en MySQL o pueden ser creadas por el usuario.

Importante

Las funciones en MySQL son muy útiles para simplificar y optimizar el código SQL, ya que permiten encapsular la lógica de una tarea en un bloque de código reutilizable y fácil de mantener, permitiendo mejorar el rendimiento al minimizar la cantidad de código repetitivo y reducir el tiempo necesario para procesar las consultas.

5.2 Funciones predefinidas

Las funciones predefinidas de MySQL incluyen funciones matemáticas, de fecha y hora, de cadena, de control de flujo y de agregación, entre otras.

Esta es una relación no exhaustiva de algunas de las funciones predefinidas de que dispone MySQL:

Función	Descripción y ejemplo
COUNT()	Devuelve el número de filas que coinciden con una consulta. SELECT COUNT(*) FROM clientes;
SUM()	Devuelve la suma de los valores en una columna. SELECT SUM(precio) FROM productos;
AVG()	Devuelve el valor promedio en una columna. SELECT AVG(edad) FROM usuarios;
MIN()	Devuelve el valor mínimo en una columna. SELECT MIN(precio) FROM productos;

MAX()	Devuelve el valor máximo en una columna. SELECT MAX(edad) FROM usuarios;
CONCAT()	Concatena dos o más cadenas de caracteres. SELECT CONCAT(nombre, ' ', apellido) AS nombre_completo FROM clientes;
SUBSTR()	Devuelve una subcadena de una cadena de caracteres. SELECT SUBSTR(nombre, 1, 3) FROM clientes;
UPPER()	Convierte una cadena de caracteres en mayúsculas. SELECT UPPER(nombre) FROM clientes;
LOWER()	Convierte una cadena de caracteres en minúsculas. SELECT LOWER(nombre) FROM clientes;
IF()	Devuelve un valor si se cumple una condición y otro valor si no se cumple. SELECT IF(edad > 18, 'Adulto', 'Menor') FROM usuarios;
CURDATE() **CURRENT_DATE()**	Devuelve la fecha actual. SELECT CURDATE();
NOW()	Devuelve la fecha y hora actuales. SELECT NOW();
RAND()	Devuelve un número aleatorio. SELECT RAND();
LENGTH()	Devuelve la longitud de una cadena de caracteres. SELECT LENGTH(nombre) FROM clients;
TRIM()	Elimina los espacios en blanco al principio y al final de una cadena de caracteres. SELECT TRIM(nombre) FROM clientes;
ROUND()	Redondea un valor a un número especificado de decimales. SELECT ROUND(123.456, 2); Este ejemplo redondea el valor 123.456 a 2 decimales y devuelve 123.46.
CEIL()	Redondea un valor hacia arriba al entero más cercano. SELECT CEIL(2.1); Este ejemplo redondea el valor 2.1 hacia arriba al entero más cercano y devuelve 3.

FLOOR()	Redondea un valor hacia abajo al entero más cercano. SELECT FLOOR(2.9); Este ejemplo redondea el valor 2.9 hacia abajo al entero más cercano y devuelve 2.
DATEDIFF()	Devuelve la diferencia entre dos fechas en días. SELECT DATEDIFF('2024-01-01', '2023-12-01'); Este ejemplo devuelve la diferencia en días entre las fechas del 1 de diciembre de 2023 y del 1 de enero de 2024, lo que da como resultado 31 días.
CONCAT_WS()	Concatena dos o más cadenas de caracteres con un separador específico. SELECT CONCAT_WS('-', '2023', '04', '08'); Este ejemplo concatena las cadenas de caracteres "2023", "04" y "08" con un guion (-) como separador y devuelve "2023-04-08".

Además de las funciones predefinidas, MySQL permite a los usuarios crear sus propias funciones personalizadas utilizando las palabras reservadas que vimos en apartados anteriores.

Las funciones personalizadas pueden ser utilizadas para realizar tareas complejas y específicas que no se pueden realizar fácilmente con las funciones predefinidas.

5.3. Funciones personalizadas

Las funciones personalizadas, también conocidas como funciones definidas por el usuario, son funciones que los usuarios pueden crear en MySQL para realizar tareas específicas que no están cubiertas por las funciones predefinidas de MySQL.

Se utilizan a menudo para simplificar consultas complejas o realizar cálculos personalizados, pudiendo también ser útiles para automatizar tareas y mejorar el rendimiento de una base de datos.

Se crean utilizando la sintaxis **CREATE FUNCTION** y se pueden llamar como cualquier otra función predefinida de MySQL. Estas funciones pueden tomar uno o varios argumentos y devolver un valor o conjunto de valores. El cuerpo de la función tiene el código que realiza la tarea específica.

Una vez creada la función personalizada, se puede llamar utilizando la sintaxis **SELECT** o como parte de una consulta más compleja.

Importante

Las funciones personalizadas pueden afectar el rendimiento de una base de datos si se usan incorrectamente o se ejecutan con demasiada frecuencia, por lo que es recomendable utilizarlas con cuidado.

5.3.1. La importancia de la sentencia DELIMITER

En MySQL, **DELIMITER** es una instrucción utilizada para cambiar el delimitador de comandos que se utiliza en una sesión de MySQL.

El delimitador predeterminado de comandos es el punto y coma (**;**), que se utiliza para separar las instrucciones SQL individuales. Sin embargo, cuando se escriben procedimientos almacenados, funciones o desencadenadores en MySQL, es necesario utilizar instrucciones SQL más complejas que contienen puntos y comas en su sintaxis.

En este caso, si se utiliza el delimitador predeterminado, MySQL interpretará cada punto y coma como el final del procedimiento o función almacenada, y generará un error de sintaxis.

Para evitar esto, se puede cambiar temporalmente el delimitador de comandos utilizando la instrucción **DELIMITER**, que permitirá escribir y almacenar el procedimiento o función almacenada correctamente.

Por ejemplo, si se quisiera crear una función almacenado que seleccione y muestre algunos datos de una tabla, la sintaxis podría ser la siguiente:

Ejemplo:

```
DELIMITER //
CREATE FUNCTION saludo()
RETURNS varchar(15)
DETERMINISTIC
BEGIN
   RETURN 'Hola que tal';
END //
DELIMITER ;
```

En este ejemplo, se utiliza **DELIMITER** para cambiar el delimitador de comandos de (;) a (//), y luego se define una función.

Al final, se utiliza nuevamente DELIMITER para volver al delimitador de comandos predeterminado (;).

De esta manera, se evita que MySQL interprete el punto y coma dentro de la función como el final del mismo, lo que generaría un error de sintaxis.

5.3.2 Etiquetado de las funciones personalizadas

En MySQL, las funciones pueden ser etiquetadas como **DETERMINISTIC**, **NO SQL** o **READS SQL DATA** para indicar cómo afectan a los datos y si son deterministas o no deterministas. Estas etiquetas se utilizan para mejorar la eficiencia de la consulta y evitar problemas de replicación de bases de datos.

Función	Descripción y ejemplo
DETERMINISTIC	Indica que la función siempre devuelve el mismo resultado para los mismos argumentos de entrada. Esto significa que la función no produce efectos secundarios y es segura para la replicación de bases de datos. Ejemplo: DELIMITER //

	CREATE FUNCTION getValorDoble(x INT) RETURNS INT DETERMINISTIC BEGIN RETURN x * 2; END // DELIMITER ;

NO SQL	Indica que la función no accede a la base de datos. Esto significa que la función no lee ni modifica ninguna tabla o vista de la base de datos.
	
	Ejemplo:
	
	DELIMITER //
	
	CREATE FUNCTION getCurrentDate() RETURNS DATE NO SQL
	BEGIN
	  DECLARE currentDate DATE;
	  SET currentDate = CURDATE();
	  RETURN currentDate;
	END
	//
	DELIMITER ;

READS SQL DATA	Indica que la función accede a los datos en la base de datos, pero no modifica los datos. Esto significa que la función solo lee los datos y no los cambia.
	
	Ejemplo:
	
	DELIMITER //
	
	CREATE FUNCTION getCustomerName(customerId INT) RETURNS VARCHAR(50) READS SQL DATA
	BEGIN
	  DECLARE customerName VARCHAR(50);
	  SELECT name INTO customerName FROM customers WHERE id = customerId;
	  RETURN customerName;
	END
	//
	DELIMITER ;

Importante

Estas etiquetas ayudan a mejorar la eficiencia de la consulta y evitar problemas de replicación de bases de datos.

Se puede dar el caso de que una misma función, puede tener asociada más de una etiqueta.

En MySQL, la opción "**log_bin_trust_function_creators**" es utilizada para permitir que las funciones definidas por el usuario que modifican datos sean replicadas en la base de datos secundaria.

Al establecer esta opción en 1 con el comando "**SET GLOBAL**", se permite que los usuarios que crean funciones puedan también modificar datos y que estos cambios se repliquen en todas las bases de datos.

```
SET GLOBAL log_bin_trust_function_creators = 1;
```

Aunque no se recomienda por seguridad, esta configuración permite no tener que indicar las etiquetas anteriores en las funciones creadas.

5.3.3 Ejemplos de funciones personalizadas

A continuación, vamos a ver varios ejemplos de funciones personalizadas:

Ejemplo 1 de función personalizada: DTERMINISTIC y NO SQL:

En el siguiente ejemplo, una función personalizada devuelve la longitud de una cadena:

```
DELIMITER //
-- Crear la función:
CREATE FUNCTION string_length2(str VARCHAR(255))
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
 -- Devuelve el número de caracteres, no de bytes:
```

```
  RETURN CHAR_LENGTH(str);
END//
DELIMITER ;

-- Ejecutar la función:
SELECT string_length2('Hola, ¡Mundo!');  -- devuelve 13
```

En este ejemplo, la función predefinida "LENGTH" se utiliza para determinar la longitud de la cadena que se pasa como parámetro.

La función personalizada "longitud_cadena" devuelve la longitud de la cadena utilizando la función predefinida de MySQL.

La función se define utilizando la sintaxis **CREATE FUNCTION** y se llama utilizando la sintaxis **SELECT**.

Ejemplo 2 de función personalizada: DTERMINISTIC y NO SQL:

En este otro ejemplo, crearemos y usaremos una función personalizada que toma dos números como argumentos y devuelve su suma:

```
DELIMITER //
#Crear la función:
CREATE FUNCTION sumar(a INT, b INT)
RETURNS INT
DETERMINISTIC NO SQL
BEGIN
   DECLARE resultado INT;
   SET resultado = a + b;
   RETURN resultado;
END
//
DELIMITER ;


#Ejecutamos la función:
SELECT sumar(10, 5) as Resultado;
#El resultado de esta consulta será 15, que es la suma de los números 10 y 5.
```

En este ejemplo, la función personalizada "**sumar**" toma dos números como argumentos y los suma utilizando la sentencia **SET**.
La función devuelve el resultado utilizando la sentencia **RETURN**.
La función se define utilizando la sintaxis **CREATE FUNCTION** y se llama utilizando la sintaxis **SELECT**.

Ejemplo 3 de función personalizada: READS SQL DATA

```
#Para que se entienda mejor el ejemplo, creamos una tabla e insertamos datos:
CREATE TABLE ventas (
   id INT AUTO_INCREMENT PRIMARY KEY,
   fecha_venta DATE NOT NULL,
   producto VARCHAR(50) NOT NULL,
   cantidad_vendida INT NOT NULL,
   precio_unitario DECIMAL(10, 2) NOT NULL,
   total DECIMAL(10, 2) NOT NULL
);

INSERT INTO ventas (fecha_venta, producto, cantidad_vendida, precio_unitario, total)
VALUES
     ('2022-01-01', 'Producto A', 10, 25.50, 255.00),
     ('2022-01-02', 'Producto B', 5, 12.75, 63.75),
     ('2022-01-03', 'Producto C', 2, 50.00, 100.00),
     ('2022-01-05', 'Producto A', 8, 25.50, 204.00),
     ('2022-01-07', 'Producto D', 3, 75.00, 225.00),
     ('2022-01-10', 'Producto B', 7, 12.75, 89.25),
     ('2022-01-12', 'Producto C', 5, 50.00, 250.00),
     ('2022-01-15', 'Producto A', 4, 25.50, 102.00),
     ('2022-01-17', 'Producto B', 9, 12.75, 114.75),
     ('2022-01-20', 'Producto D', 6, 75.00, 450.00);

#Ahora creamos la función que recibirá como parámetro el mes del que queremos filtrar la cantidad vendida, retornando el acumulado vendido para dicho mes:
DELIMITER //
CREATE FUNCTION ventas_mensuales (mes INT)
RETURNS INT
READS SQL DATA
BEGIN
   DECLARE cantidad INT;
```

```
  SELECT SUM(cantidad_vendida) INTO cantidad FROM ventas WHERE MONTH(fecha_venta) = mes;
  RETURN cantidad;
END;
//

DELIMITER ;

#Llamamos a la función ventas_mensuales, indicando el mes de enero, y obteniendo 59 como resultado:
SELECT ventas_mensuales(1);
```

6. Estructura de control de flujo (disyunción, iteración)

En MySQL, existen estructuras de control de flujo que permiten controlar la ejecución de las instrucciones SQL en función de ciertas condiciones o repetir un conjunto de instrucciones varias veces.

Las principales estructuras de control de flujo en MySQL son la disyunción y la iteración, que se describen a continuación:

Disyunción: IF	Se implementa a través de la instrucción **IF**.
	
	La sintaxis es la siguiente:
	
	IF condición1 THEN
	  -- conjunto de instrucciones SQL a ejecutar si la condición1 es verdadera
	ELSEIF condición2 THEN
	  -- conjunto de instrucciones SQL a ejecutar si la condición2 es verdadera
	ELSEIF condición3 THEN
	  -- conjunto de instrucciones SQL a ejecutar si la condición3 es verdadera
	…
	ELSE
	  -- conjunto de instrucciones SQL a ejecutar si ninguna de las condiciones anteriores es verdadera
	END IF;
	
	En este caso, se especifica una condición que se evalúa como verdadera o falsa, y se ejecuta un conjunto de instrucciones SQL si la condición es verdadera y otro conjunto de instrucciones SQL si la condición es falsa.

Iteración: WHILE	La iteración en MySQL se implementa a través de la instrucción WHILE. La sintaxis básica es la siguiente:
	
	WHILE condición DO
	  -- instrucciones a ejecutar mientras la condición es verdadera
	END WHILE;
	
	En este caso, se especifica una condición que se evalúa como verdadera o falsa, y se ejecuta un conjunto de instrucciones SQL mientras la condición es verdadera. Es importante tener

	en cuenta que la condición debe cambiar en algún momento para evitar una iteración infinita. También existe la estructura de iteración REPEAT, que es similar a WHILE pero se evalúa al final del ciclo en lugar de al principio. La sintaxis básica es la siguiente: **lua** **Copy code** **REPEAT** -- instrucciones a ejecutar mientras la condición sea falsa UNTIL condición; En este caso, se especifica un conjunto de instrucciones SQL que se ejecuta mientras la condición es falsa, y la condición se evalúa al final del ciclo. Esto significa que el conjunto de instrucciones SQL se ejecuta al menos una vez, incluso si la condición es falsa desde el principio.
Iteración: REPAT	También existe la estructura de iteración **REPEAT**, que es similar a WHILE pero se evalúa al final del ciclo en lugar de al principio. La sintaxis básica es la siguiente: REPEAT -- instrucciones a ejecutar mientras la condición es verdadera UNTIL condición; END REPEAT En este caso, se especifica un conjunto de instrucciones SQL que se ejecuta mientras la condición es falsa, y la condición se evalúa al final del ciclo. Esto significa que el conjunto de instrucciones SQL se ejecuta al menos una vez, incluso si la condición es falsa desde el principio.

6.1. Disyunción: IF

Ejemplo IF:

```
#Creamos una función que incorpore lógica de disyunción:
DELIMITER //

CREATE FUNCTION tipoImporte(importe int)
RETURNS varchar(50)
DETERMINISTIC

BEGIN
SET @mensaje  = '';

IF importe < 1000 THEN
   SET @mensaje = 'Menor a 1000';
ELSEIF importe < 2000 THEN
   SET @mensaje = 'Menor a 2000';
ELSEIF importe < 3000 THEN
   SET @mensaje = 'Menor a 3000';
ELSE
   SET @mensaje = 'Igual o superior a 3000';
END IF;

RETURN @mensaje;
END
//
DELIMITER ;

#Ejecutamos la función pásandole 3000 como parámetro:
SELECT tipoImporte(3000);
```

```
DELIMITER //

CREATE FUNCTION tipoImporte(importe INT)
 RETURNS VARCHAR(50)
 DETERMINISTIC
BEGIN
 -- Variable local para almacenar el mensaje de retorno
 DECLARE mensaje VARCHAR(50) DEFAULT '';
```

```
  IF importe < 1000 THEN
    SET mensaje = 'Menor a 1000';
  ELSEIF importe < 2000 THEN
    SET mensaje = 'Menor a 2000';
  ELSEIF importe < 3000 THEN
    SET mensaje = 'Menor a 3000';
  ELSE
    SET mensaje = 'Igual o superior a 3000';
  END IF;

  RETURN mensaje;
END//

DELIMITER ;

-- Prueba de la función:
SELECT tipoImporte(3000);  -- devuelve 'Igual o superior a 3000'
```

Este código es un ejemplo de creación de una función en MySQL que recibe un parámetro (importe) y devuelve un mensaje indicando en qué rango se encuentra ese importe.

En la primera línea, se establece un nuevo delimitador con **DELIMITER //**, lo que significa que la finalización de una instrucción SQL se indica ahora con **//** en lugar de ; Esto se hace para poder incluir instrucciones compuestas en la función, como la estructura **IF** que se utiliza a continuación.
La función se llama tipo Importe y recibe un parámetro **importe** de tipo **int** y devuelve una cadena de texto de hasta 50 caracteres.

La cláusula **DETERMINISTIC** se utiliza para indicar que la función siempre devuelve el mismo resultado para un conjunto dado de argumentos.
Dentro del bloque **BEGIN** y **END**, se define la lógica de la función. Se establece una variable **@mensaje** que se utilizará para almacenar el mensaje de retorno.

A continuación, se utiliza una estructura de control de flujo condicional **IF** con la cláusula **ELSEIF** para comprobar el valor del parámetro importe y establecer el valor correspondiente para la variable **@mensaje**.
Si el importe es menor a 1000, se establece el mensaje "Menor a 1000". Si es mayor o igual a 1000 y menor a 2000, se establece "Menor a 2000", y así sucesivamente.

Finalmente, se devuelve el valor de la variable @mensaje con la instrucción **RETURN**.

Para finalizar la definición de la función, se establece de nuevo el delimitador a **;** con **DELIMITER ;**.
Por último, se realiza una consulta utilizando la función **tipo Importe**, pasando como argumento el valor **3000**. La consulta devuelve la cadena "Igual o superior a 3000", que es el mensaje correspondiente para ese valor de importe según la lógica definida en la función.

6.2. Iteración: WHILE

Ejemplo WHILE:

```
DELIMITER //
CREATE FUNCTION factorial(n INT)
RETURNS INT
DETERMINISTIC
BEGIN
  DECLARE result INT DEFAULT 1;
  DECLARE i INT DEFAULT 1;

  WHILE i <= n DO
    SET result = result * i;
    SET i = i + 1;
  END WHILE;

  RETURN result;
```

```
END //
DELIMITER;
```

En este ejemplo, la función se llama factorial y toma un argumento n de tipo INT. Devuelve el resultado del cálculo de la factorial de n.

Dentro del bloque BEGIN y END, se definen dos variables locales: result y i. La variable result se inicializa en 1 porque la factorial de cualquier número multiplicado por 1 es igual al propio número, y i se inicializa en 1 para comenzar el bucle.

A continuación, se utiliza un bucle WHILE que se ejecutará mientras i sea menor o igual que n.

En cada iteración del bucle, se multiplica el valor actual de result por i y se almacena el resultado en result. Además, se incrementa el valor de i en 1 para avanzar al siguiente número en el cálculo de la factorial.

Después del bucle, se devuelve el valor final de result con la instrucción RETURN.

Con este ejemplo hemos visto cómo usar un bucle WHILE dentro de una función en MySQL para realizar cálculos repetitivos y devolver un resultado.

6.3. Iteración: REPEAT

Ejemplo REPEAT:

```
DELIMITER //
CREATE FUNCTION factorial(n INT)
RETURNS INT DETERMINISTIC
BEGIN
  DECLARE result INT DEFAULT 1;
  DECLARE i INT DEFAULT 1;

  REPEAT
    SET result = result * i;
    SET i = i + 1;
  UNTIL i > n
```

```
END REPEAT;

  RETURN result;
END //

DELIMITER;
```

En este ejemplo, la estructura del bucle REPEAT es similar al bucle WHILE, pero tiene una sintaxis ligeramente diferente.
Dentro del bloque BEGIN y END, se definen dos variables locales: result e i. La variable result se inicializa en 1 y i se inicializa en 1 para comenzar el bucle.

A continuación, se utiliza un bucle REPEAT que se ejecutará hasta que i sea mayor que n. En cada iteración del bucle, se multiplica el valor actual de result por i y se almacena el resultado en result. Además, se incrementa el valor de i en 1 para avanzar al siguiente número en el cálculo de la factorial.
Después del bucle, se devuelve el valor final de result con la instrucción RETURN.

Este ejemplo muestra cómo utilizar un bucle REPEAT dentro de una función en MySQL para realizar cálculos repetitivos y devolver un resultado. Ambas estructuras de bucle, WHILE y REPEAT, son útiles en diferentes situaciones y ofrecen formas alternativas de implementar iteraciones en MySQL.

Veamos, mediante ejemplo práctico, en qué caso se puede aplicar el bucle:

FitLife, que como ya sabemos es un gimnasio de barrio, quiere poner en marcha un programa de fidelización para sus socios. Cada mes el sistema registra el número de visitas de cada socio en la tabla asistencia:

```
CREATE TABLE asistencia (
 socio_id   INT,
 mes        CHAR(7),      -- 'YYYY-MM'
 visitas    INT,
 PRIMARY KEY(socio_id, mes)
);
```

Para premiar la constancia, nos piden:

1. **Categorizar** el nivel de actividad mensual en "Bajo", "Medio", "Alto" o "Muy Alto" según número de visitas, usando IF.
2. **Calcular puntos de bonificación:** por cada bloque completo de 5 visitas, el socio recibe 2 puntos extra; hacerlo con un bucle WHILE.
3. **Determinar pases gratis**: por cada 20 puntos acumulados, el socio recibe un pase de invitado; implementarlo con REPEAT.

Preguntas

1. Escribe la función categoria_visitas(visitas INT) que devuelva:
   - "Bajo" si visitas < 5
   - "Medio" si 5 <= visitas < 10
   - "Alto" si 10 <= visitas < 20
   - "Muy Alto" en caso contrario.
2. Escribe la función puntos_bono(visitas INT) que use un bucle WHILE para calcular 2 * (visitas DIV 5).
3. Escribe la función pases_gratis(puntos INT) que use un bucle REPEAT para restar de 20 en 20 hasta que queden menos de 20 puntos, y devuelva el número de veces que pudo restar (cada restada genera un pase gratis).

Solución

1) Función con IF:

```
DELIMITER //
CREATE FUNCTION categoria_visitas(visitas INT)
 RETURNS VARCHAR(10)
 DETERMINISTIC
BEGIN
 DECLARE cat VARCHAR(10) DEFAULT '';

 IF visitas < 5 THEN
```

```
    SET cat = 'Bajo';
  ELSEIF visitas < 10 THEN
    SET cat = 'Medio';
  ELSEIF visitas < 20 THEN
    SET cat = 'Alto';
  ELSE
    SET cat = 'Muy Alto';
  END IF;

  RETURN cat;
END//
DELIMITER ;

-- Ejemplos:
SELECT categoria_visitas(3)  AS Nivel;  -- 'Bajo'
SELECT categoria_visitas(12) AS Nivel;  -- 'Alto'
```

2) Función con WHILE:

```
DELIMITER //
CREATE FUNCTION puntos_bono(visitas INT)
  RETURNS INT
  DETERMINISTIC
BEGIN
  DECLARE puntos INT DEFAULT 0;
  DECLARE bloques INT;

  -- Cada bloque de 5 visitas vale 2 puntos
  SET bloques = visitas DIV 5;
```

```
  WHILE bloques > 0 DO
    SET puntos = puntos + 2;
    SET bloques = bloques - 1;
  END WHILE;

  RETURN puntos;
END//
DELIMITER ;

-- Ejemplos:
SELECT puntos_bono(3)  AS Bono;  -- 0  (0 bloques)
SELECT puntos_bono(12) AS Bono;  -- 4  (2 bloques)
```

3) Función con REPEAT:

```
DELIMITER //
CREATE FUNCTION pases_gratis(puntos INT)
  RETURNS INT
  DETERMINISTIC
BEGIN
  DECLARE pases INT DEFAULT 0;

  REPEAT
    SET pases = pases + 1;
    SET puntos = puntos - 20;
  UNTIL puntos < 20
  END REPEAT;

  RETURN pases;
END//
```

```
DELIMITER ;
```

Ejemplos:

```
SELECT pases_gratis(15)  AS Pases; -- 0 (nunca llega a 20)
SELECT pases_gratis(45)  AS Pases; -- 2 (45-20=25 → 1 pase, 25-20=5 → 2 pases)
```

```
-- Caso práctico: combinar todo para un mes y un socio
-- Supongamos que quiero un informe para el socio 123 en '2025-07'
SELECT
  a.socio_id,
  a.mes,
  a.visitas,
  categoria_visitas(a.visitas)   AS NivelActividad,
  puntos_bono(a.visitas)         AS PuntosBono,
  pases_gratis(puntos_bono(a.visitas)) AS PasesInvitado
FROM asistencia AS a
WHERE a.socio_id = 123
  AND a.mes = '2025-07';
```

Interpretación del informe

- NivelActividad: categoría según número de visitas.
- PuntosBono: puntos extra obtenidos.
- PasesInvitado: número de pases gratis que se puede regalar.

7. Procedimientos almacenados

7.1. ¿Qué es un procedimiento almacenado?

Un Procedimiento almacenado es un conjunto de instrucciones SQL que se guardan en la base de datos y se pueden llamar desde una aplicación o desde otros procedimientos almacenados.

En MySQL podemos distinguir dos tipos de procedimientos almacenados:

- **Funciones:** Son procedimientos almacenados que devuelven un valor, y que vimos en apartados anteriores.
- **Procedimientos:** son procedimientos almacenados que no devuelven un valor, y que desarrollaremos en este capítulo.

7.2 Características de los procedimientos almacenados

Reutilización de código	Se pueden llamar desde diferentes lugares de la aplicación o desde otros procedimientos almacenados.
Seguridad	Permiten controlar quién puede **ejecutar** cada procedimiento almacenado mediante permisos granulares, restringiendo así el acceso a operaciones sensibles.
Rendimiento	Se compilan una sola vez y se almacenan en la base de datos, lo que mejora el rendimiento al reducir el tiempo de compilación y de comunicación entre la aplicación y la base de datos.
Modularidad	Permiten la modularización del código, lo que facilita su mantenimiento y evolución.
Transacciones	Se pueden utilizar en transacciones, lo que garantiza la consistencia de los datos.
Depuración	Se pueden "trazar" con SELECT/SIGNAL o usar herramientas externas (MySQL Workbench, etc.) para localizar errores.

7.3 Diferencia entre funciones y procedimientos almacenados

Antes de continuar, vamos a enumerar las diferencias que hay entre Funciones y Procedimientos almacenados, más allá de la más evidente, que es que las Funciones siempre devuelven un valor y los Procedimientos almacenados no lo hacen.

- **Funciones:**
  - Devuelven un valor, que puede ser de cualquier tipo de datos permitido en MySQL.
  - Se pueden utilizar en expresiones SQL, es decir, se pueden usar en cualquier lugar donde se pueda usar una expresión SQL válida.
  - Las funciones de MySQL pueden usarse en cualquier expresión SQL válida (por ejemplo, en la lista de columnas de un SELECT, en condiciones WHERE o HAVING, en cláusulas SET, en JOIN … ON, en ORDER BY, etc.).
  - No pueden modificar los datos de la base de datos, sólo pueden leerlos.
  - No pueden contener sentencias que modifiquen la estructura de la base de datos, como la creación o eliminación de tablas.

- **Procedimientos:**
  - No devuelven ningún valor, aunque pueden generar valores como resultado de su ejecución mediante parámetros de salida.
  - Se llaman con una sentencia **CALL**.
  - Pueden modificar los datos de la base de datos.
  - Pueden contener sentencias que modifiquen la estructura de la base de datos, como la creación o eliminación de tablas.
  - Se pueden utilizar para encapsular la lógica de negocio, dividir un proceso complejo en pasos más simples, mejorar el rendimiento, la seguridad, y la modularidad del código.

7.4 Estructura y ejecución de los procedimientos almacenados

Como dijimos anteriormente, los procedimientos almacenados en MySQL son un conjunto de instrucciones SQL que se guardan en la base de datos y se

pueden llamar desde una aplicación o desde otros procedimientos almacenados.

7.4.1. Creación de procedimientos almacenados

Para **crear** un procedimiento almacenado en MySQL, se utiliza la siguiente sintaxis:

```
DELIMITER //

CREATE [DEFINER = user] PROCEDURE Procedure_name ([Parameter_list])
BEGIN
   -- cuerpo del procedimiento
END;

DELIMITER ;
```

Donde:

- **DEFINER**: Especifica el usuario que crea el procedimiento. Si no se especifica, el usuario actual será el definidor del procedimiento.
- **Procedure_name**: Es el nombre del procedimiento almacenado.
- **Parameter_list**: Es una lista opcional de parámetros que se pueden pasar al procedimiento.
- **BEGIN** y **END**: Delimitan el cuerpo del procedimiento, que contiene las instrucciones SQL que se ejecutarán cuando se llame al procedimiento.

En MySQL los parámetros no son "anónimos": hay que declarar si entran (IN), salen (OUT) o ambas cosas (INOUT). Ejemplo:

```
DELIMITER //

CREATE [DEFINER = 'usuario'@'origen'] PROCEDURE nombre_procedimiento(

  IN  param1 INT,

  OUT param2 VARCHAR(20)

)

BEGIN

  -- cuerpo del procedimiento…

END//
```

```
DELIMITER ;

-- Para invocarlo:

CALL nombre_procedimiento(123, @salida);
```

El **modo de parámetro** (IN, OUT, INOUT) es obligatorio si vas a recibir o devolver valores.

7.4.2. Ejecución de procedimientos almacenados

Para llamar a un procedimiento almacenado en MySQL, se utiliza la siguiente sintaxis:

```
CALL Procedure_name([Parameter_list]);
```

Donde:

- **Procedure_name**: Es el nombre del procedimiento almacenado.
- **Parameter_list**: Es una lista opcional de parámetros que se pueden pasar al procedimiento.

7.5. Ejemplos de procedimientos almacenado

7.5.1 Procedimiento almacenado

Un ejemplo de procedimiento almacenado simple en MySQL es el siguiente:

```
DELIMITER //

CREATE PROCEDURE my_procedure()

BEGIN

  SELECT * FROM my_table;

END;

//

DELIMITER ;
```

Este procedimiento almacenado simplemente selecciona todos los registros de la tabla **my_table** y los devuelve como resultado.

Un ejemplo de llamada a un procedimiento almacenado en MySQL es el siguiente:

```
CALL my_procedure();
```

Este código llama al procedimiento almacenado **my_procedure** y devuelve todos los registros de la tabla **my_table**.

7.5.2 Procedimiento almacenado con lista de parámetros

```
-- Este procedimiento almacenado actualiza la nota de un estudiante en una asignatura
-- determinada, recibiendo como parámetros el id del estudiante, el id de la asignatura
-- y la nueva nota a asignar.
-- Se define el procedimiento almacenado con tres parámetros de entrada
DELIMITER //

CREATE PROCEDURE update_calificacion(
  IN p_estudiante_id INT,
  IN p_asignatura_id INT,
  IN p_calificacion  INT
)
BEGIN
  -- Intentamos actualizar
  UPDATE calificaciones
  SET calificacion = p_calificacion
  WHERE estudiante_id  = p_estudiante_id
    AND asignatura_id  = p_asignatura_id;
  -- Si no existía, insertamos
  IF ROW_COUNT() = 0 THEN
    INSERT INTO calificaciones(estudiante_id, asignatura_id, calificacion)
    VALUES (p_estudiante_id, p_asignatura_id, p_calificacion);
```

```
 END IF;
END//
DELIMITER ;
```

En este ejemplo, el procedimiento almacenado recibe tres parámetros de entrada:

- estudiante_id
- asignatura_id
- calificacion.

Estos parámetros representan el identificador del estudiante, el identificador de la asignatura y la nueva nota a asignar, respectivamente.

El procedimiento almacenado actualiza la tabla calificaciones con la nueva nota para el estudiante y la asignatura indicados.

Si no se encuentra ningún registro correspondiente en la tabla calificaciones, se inserta un nuevo registro con los valores proporcionados en los parámetros.

No se utilizan cursores en este ejemplo que veremos más adelante.

Podemos probar este procedimiento creando previamente la siguiente tabla:

```
CREATE TABLE calificaciones (
   id INT AUTO_INCREMENT PRIMARY KEY,
   estudiante_id INT NOT NULL,
   asignatura_id INT NOT NULL,
   calificacion INT NOT NULL,
);
```

7.5.3 Procedimiento almacenado con lista de parámetros y mensaje de salida

```
-- Este procedimiento almacenado actualiza la nota de un estudiante en una asignatura
-- determinada, recibiendo como parámetros el id del estudiante, el id de la asignatura
```

```
-- y la nueva nota a asignar.
-- Se define el procedimiento almacenado con tres parámetros de entrada
DELIMITER //
CREATE PROCEDURE update_calificacion_con_mensaje(IN estudiante_id INT, IN asignatura_id INT, IN calificacion INT)
BEGIN

  -- Se actualiza la nota del estudiante en la asignatura indicada
  UPDATE calificaciones
  SET calificacion = calificacion
  WHERE estudiante_id = estudiante_id AND asignatura_id = asignatura_id;

  -- Se verifica si se afectó alguna fila
  IF (ROW_COUNT() = 0) THEN

    -- Si no se afectó ninguna fila, se inserta un nuevo registro en la tabla 'calificaciones'
    INSERT INTO calificaciones(estudiante_id, asignatura_id, calificacion)
    VALUES(estudiante_id, asignatura_id, calificacion);
SELECT CONCAT('Se ha insertado una nueva nota de ', calificacion,
              ' para el estudiante con id ', estudiante_id,
              ' en la asignatura con id ', asignatura_id) AS 'Mensaje';
  ELSE
    SELECT CONCAT('Se ha actualizado la nota del estudiante con id ', estudiante_id,
              ' en la asignatura con id ', asignatura_id,
              ' a la nueva nota de ', calificacion) AS 'Mensaje';
  END IF;
END;
```

```
//
DELIMITER ;
```

Este procedimiento almacenado es igual que la del ejemplo del apartado 7.5.2, pero incluye una salida en forma de mensaje en función de si se ha actualizado la nota o se ha agregado.

7.5.4. Procedimiento almacenado llamado desde otro procedimiento almacenado

```
-- Este procedimiento almacenado llama a otro procedimiento. En este caso, llamamos al procedimiento almacenado del ejemplo 7.5.3
DELIMITER //

CREATE PROCEDURE llama_a_otro_procedimiento(
  IN estudiante_id INT,
  IN asignatura_id INT,
  IN calificacion   INT
)
BEGIN
  CALL update_calificacion_con_mensaje(
    estudiante_id,
    asignatura_id,
    calificacion
  );
END//

DELIMITER ;
```

En el siguiente ejemplo veremos cómo aplicar DELIMITER - Gestión de niveles de socio y reseteo mensual en FitLife

El gimnasio FitLife quiere automatizar la asignación del nivel de socio (Bronze, Silver, Gold) en función del número de asistencias de cada miembro durante el último mes, y además resetear a cero el contador de asistencias al inicio de cada mes. Para ello, dispone de dos tablas:

- **members**
  - id INT PK
  - name VARCHAR(50)
  - status ENUM('Bronze','Silver','Gold') DEFAULT 'Bronze'
- **attendances**
  - id INT PK
  - member_id INT FK → members(id)
  - att_date DATE

Preguntas a resolver

1. Crear un procedimiento almacenado award_status que, dado el identificador de un socio, calcule cuántas veces ha asistido en los últimos 30 días y actualice su campo status según:
   - 0–4 asistencias → Bronze
   - 5–15 asistencias → Silver
   - ≥16 asistencias → Gold
2. Crear un segundo procedimiento process_all_members() que recorra todos los socios y, para cada uno, invoque award_status, de modo que actualice masivamente el estado de todos los miembros.
3. Crear un tercer procedimiento monthly_reset() que primero borre todas las filas de attendances con fecha anterior al primer día del mes en curso (limpieza mensual), y luego invoque process_all_members() para recalcular los estados sobre las asistencias del nuevo mes.

Solución (código SQL)

1) Procedimiento: award_status

```
DELIMITER //

CREATE PROCEDURE award_status(IN p_member_id INT)
BEGIN
  DECLARE cnt INT DEFAULT 0;

  -- Contar asistencias de los últimos 30 días
  SELECT COUNT(*) INTO cnt
    FROM attendances
    WHERE member_id = p_member_id
      AND att_date >= CURDATE() - INTERVAL 30 DAY;

  -- Asignar status según número de asistencias
  IF cnt < 5 THEN
    UPDATE members SET status = 'Bronze' WHERE id = p_member_id;
  ELSEIF cnt < 16 THEN
    UPDATE members SET status = 'Silver' WHERE id = p_member_id;
  ELSE
    UPDATE members SET status = 'Gold'   WHERE id = p_member_id;
  END IF;
END;
//

DELIMITER ;
```

2) Procedimiento: process_all_members

```
DELIMITER //

CREATE PROCEDURE process_all_members()
BEGIN
  DECLARE done  BOOL DEFAULT FALSE;
  DECLARE m_id   INT;
  DECLARE cur    CURSOR FOR SELECT id FROM members;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

  OPEN cur;
  read_loop: LOOP
    FETCH cur INTO m_id;
    IF done THEN
      LEAVE read_loop;
    END IF;
    CALL award_status(m_id);
  END LOOP;
  CLOSE cur;
END;
//

DELIMITER ;
```

3) Procedimiento: monthly_reset

```
DELIMITER //

CREATE PROCEDURE monthly_reset()
BEGIN
```

```
  -- Eliminar asistencias de meses anteriores
  DELETE FROM attendances
  WHERE att_date < DATE_SUB(CURDATE(), INTERVAL DAYOFMONTH(CURDATE())-1 DAY);

  -- Recalcular status de todos los socios
  CALL process_all_members();
END;
//

DELIMITER ;
```

Con estos tres procedimientos, FitLife Gym puede automatizar tanto la limpieza de datos antiguos, como la evaluación individual y masiva del nivel de sus socios cada mes.

8. Subrutinas. Ámbito de variables

8.1 Subrutinas

En MySQL, un subprograma o subrutina es un bloque de código que puede ser llamado desde otros lugares en una base de datos MySQL.

Los subprogramas permiten a los desarrolladores escribir código reutilizable, lo que puede simplificar el proceso de desarrollo y reducir la cantidad de código que se necesita escribir para realizar ciertas tareas, pudiendo también ayudar a mejorar el rendimiento de una base de datos MySQL, ya que el código se puede ejecutar de manera más eficiente dentro de un subprograma. Las "subrutinas" en MySQL son simplemente procedimientos o funciones definidas por el usuario; no existe un objeto separado llamado "subprograma".

En el ejemplo 7.5.4 del apartado dedicado a los procedimientos almacenados, ya vimos cómo llamar a un subprograma, aunque el mismo procedimiento es aplicable también a las funciones, usando **SELECT** para llamarlas, en lugar de **CALL**, utilizado parar llamar a los procedimientos almacenados.

Otro ejemplo: un subprograma en MySQL podría ser utilizado para realizar cálculos complejos en una tabla y devolver el resultado como un valor de retorno. Este subprograma podría ser llamado desde cualquier parte de la base de datos donde sea necesario realizar esos cálculos.

Importante

Un subprograma o subrutina en MySQL es un bloque de código reutilizable que se puede llamar desde otros lugares de la base de datos para realizar tareas específicas.

8.2 Variables globales y locales

En MySQL distinguimos dos tipos de variables de usuario según su ámbito:

- **Variables locales**

  Se declaran con DECLARE al comienzo de un bloque BEGIN…END dentro de un procedimiento, función o bloque anidado. Solo existen mientras se ejecuta ese bloque, y no pueden usarse fuera de él.

  BEGIN

DECLARE contador INT DEFAULT 0;

- aquí ya puedes usar contador…

END

- **Variables globales de usuario**

  Se identifican con @nombre y pueden crearse y leerse en cualquier parte de la sesión. Persisten hasta que se cierra la conexión, independientemente de en qué procedimiento o bloque se hayan asignado.

  SET @mi_var = 100;

  - más adelante, en otro procedimiento o consulta:

  SELECT @mi_var;

Importante

1. Todos los DECLARE de un bloque deben ir juntos **al inicio** de ese bloque, antes de cualquier otra instrucción.
2. No confundir variables locales (DECLARE) con variables globales de usuario (@var), ya que su persistencia y ámbito son diferentes.

Ejemplo de variables globales y locales:

```
DELIMITER //

CREATE PROCEDURE ejemplo_variables_globales_y_locales()
BEGIN
 -- Declaración de la "variable_global" (en realidad local al procedimiento)
 DECLARE variable_global INT DEFAULT 0;

 -- Bloque interno con su propia variable
 BEGIN
  DECLARE variable_local INT DEFAULT 0;

  SET variable_global = 10;
```

```
  SET variable_local  = 20;

  SELECT CONCAT(
   'BLOQUE INTERNO -> variable_global=', variable_global,
   ', variable_local=', variable_local
  ) AS Mensaje;
 END;

 -- Aquí solo existe variable_global; variable_local ya no está disponible
 SELECT CONCAT(
  'BLOQUE EXTERNO -> variable_global=', variable_global
 ) AS Mensaje;
END//

DELIMITER ;

-- Ejecución de prueba
CALL ejemplo_variables_globales_y_locales();
```

Este procedimiento ilustra el concepto de ámbito de variables en MySQL:

1. Se declara **variable_global** (local al procedimiento) de tipo INT y valor inicial 0.
2. Se crea un **bloque interno** donde, justo al inicio, se declara **variable_local** (local solo a ese bloque).
3. Dentro del bloque interno se asignan valores: variable_global = 10 y variable_local = 20, y a continuación se muestra un mensaje con ambos valores:

   BLOQUE INTERNO -> variable_global=10, variable_local=20

4. Al terminar el bloque interno, **variable_local** deja de existir. En el bloque exterior volvemos a concatenar y mostrar solo **variable_global**, que conserva el valor asignado en el bloque interno:

   BLOQUE EXTERNO -> variable_global=10

5. Al llamar al procedimiento, se verán dos salidas: la del bloque interno (ambas variables) y la del bloque externo (solo variable_global).

Tech Hub <FP> euroformac

9. Eventos y disparadores

En MySQL, los eventos y los disparadores son dos herramientas de programación que se utilizan para automatizar tareas en una base de datos.

Los **eventos** son objetos que se ejecutan en un momento específico o a intervalos regulares. Se utilizan para programar tareas de mantenimiento, como la actualización de tablas o la limpieza de datos. Para crear un evento, se utiliza el comando **CREATE EVENT** y se define el tiempo de ejecución y la frecuencia.

Los **disparadores**, por otro lado, son objetos que se activan automáticamente cuando se produce un evento específico, como una actualización, inserción o eliminación de datos en una tabla. Se utilizan para mantener la integridad de los datos y para aplicar reglas de negocio. Para crear un disparador, se utiliza el comando **CREATE TRIGGER** y se define el evento que lo activará, así como la acción a realizar cuando se active.

Importante

Una diferencia clave entre eventos y disparadores es que los eventos se ejecutan en un momento específico o a intervalos regulares, mientras que los disparadores se activan automáticamente cuando se produce un evento específico en una tabla.

9.1 Eventos

Para poder utilizar eventos en MySQL es imprescindible que el programador habilite el planificador de eventos del servidor. Esto se consigue ajustando la variable de sistema event_scheduler a ON, por ejemplo, ejecutando:

```
SET GLOBAL event_scheduler = ON;
```

Con esta instrucción se activa el demonio interno encargado de lanzar los eventos programados. Si event_scheduler permanece en OFF, ningún evento definido con CREATE EVENT llegará a ejecutarse, aunque exista en el catálogo de la base de datos.

Como dijimos anteriormente, los eventos son objetos de la base de datos que se ejecutan periódicamente. A continuación, vamos a ver varios ejemplos de cómo se construyen y activan, para que comprendamos mejor el concepto.

Ejemplo: copia de seguridad diaria

```
-- Activar el planificador de eventos (una sola vez):
SET GLOBAL event_scheduler = ON;

DELIMITER //

CREATE EVENT backup_event
ON SCHEDULE
  EVERY 1 DAY
  STARTS CURRENT_TIMESTAMP
DO
BEGIN
  -- Crear copia de seguridad
  INSERT INTO backup_table
    SELECT * FROM original_table;
END//

DELIMITER ;

-- Habilitar el evento (si no está habilitado por defecto)
ALTER EVENT backup_event ENABLE;
```

Este evento se llama backup_event y se ejecutará todos los días a la misma hora.

La acción que se realizará cuando se active el evento es crear una copia de seguridad de la tabla original_table y almacenarla en una nueva tabla llamada backup_table.

Para activar el evento, se puede utilizar el siguiente comando:

```
ALTER EVENT backup_event ENABLE;
```

Una vez activado, el evento se ejecutará automáticamente todos los días y creará una copia de seguridad de la tabla especificada.

Ejemplo de resumen de ventas diario:

Un ejemplo de evento para insertar periódicamente registros en una tabla resumiendo el total de ventas alcanzado podría ser el siguiente:

```
1) Activar el scheduler (una sola vez):

SET GLOBAL event_scheduler = ON;


2) (Opcional) Asegurar existencia de tabla de resumen

CREATE TABLE IF NOT EXISTS resumen_ventas (

  fecha DATE PRIMARY KEY,

  total DECIMAL(10,2)

);


DELIMITER //


CREATE EVENT resumen_ventas_diarias

  ON SCHEDULE

    EVERY 1 DAY

    STARTS CONCAT(CURDATE(), ' 00:00:00')

  DO

BEGIN

  DECLARE total_ventas_dia DECIMAL(10,2);


  -- Calcular ventas del día actual

  SELECT SUM(totalVentas)

    INTO total_ventas_dia

    FROM ventas

   WHERE fecha = CURDATE();
```

```
 -- Insertar o actualizar resumen
 INSERT INTO resumen_ventas(fecha, total)
   VALUES(CURDATE(), total_ventas_dia)
 ON DUPLICATE KEY
   UPDATE total = total_ventas_dia;
END//

DELIMITER ;
```

3) Habilitar el evento (si no lo crea como ENABLED por defecto)

```
ALTER EVENT resumen_ventas_diarias ENABLE;
```

Este evento se llama **resumen_ventas_diarias** y se ejecutará todos los días a la misma hora.

La acción que se realizará cuando se active el evento es calcular el total de ventas realizadas en el día actual y almacenarlo en una tabla de resumen de ventas llamada **resumen_ventas**.

El cálculo del total de ventas se realiza utilizando la función SUM para sumar los totales de todas las ventas registradas en la tabla ventas para el día actual. Luego, se inserta un nuevo registro en la tabla **resumen_ventas** con la fecha actual y el total de ventas calculado.

El evento lo activaremos con la sentencia:

```
ALTER EVENT resumen_ventas_diarias ENABLE;
```

9.2 Disparadores

Definición

Un **disparador** o **Trigger** en MySQL es un tipo especial de objeto que se ejecuta automáticamente en respuesta a un evento específico en una tabla, como una inserción, actualización o eliminación de filas, siendo una herramienta muy útil para la implementación de lógica de negocio compleja y para garantizar la integridad de los datos en una base de datos.

Entre las **características** de los disparadores en MySQL se encuentran:

- Se asocian a una tabla específica y se definen para activarse en respuesta a ciertos eventos que ocurren en esa tabla.
- Pueden ejecutarse antes o después de la ejecución de la acción que los desencadena.
- Permiten realizar acciones que afectan a otras tablas además de la que desencadena el evento.
- Pueden ser utilizados para implementar restricciones personalizadas y complejas en la base de datos.
- Los disparadores pueden estar escritos en lenguajes como SQL o PL/SQL.
- Pueden ser activados o desactivados según sea necesario.
- Pueden ser creados y gestionados mediante el lenguaje SQL, utilizando sentencias como **CREATE TRIGGER**, **ALTER TRIGGER** y **DROP TRIGGER**.
- MySQL sólo soporta triggers definidos con SQL procedimental interno (*MySQL stored routines*), no PL/SQL de Oracle.
- A diferencia de otros SGBD, en MySQL no puedes "activar" o "desactivar" un trigger en caliente: si no lo quieres, debes DROP TRIGGER; si quieres recuperarlo, debes CREATE TRIGGER de nuevo.

Ejemplo de disparador para usuarios y permisos:

En este ejemplo crearemos dos triggers sobre la tabla usuarios para que, cuando se inserte un registro de usuario activo, se asignen automáticamente permisos de "Lectura" y "Escritura"; y cuando se elimine un usuario, se eliminen también todos sus permisos en la tabla permisos.

Asegúrate de usar un delimitador distinto mientras defines los triggers

```
DELIMITER //

1) Trigger: después de insertar un usuario
CREATE TRIGGER permisos_after_insert
AFTER INSERT ON usuarios
FOR EACH ROW
BEGIN
  -- Solo asignamos permisos si el nuevo usuario está activo
  IF NEW.activo = 1 THEN
    INSERT INTO permisos (id_usuario, permiso)
    VALUES (NEW.id_usuario, 'Lectura'),
           (NEW.id_usuario, 'Escritura');
  END IF;
END;
//

2) Trigger: después de eliminar un usuario
CREATE TRIGGER permisos_after_delete
AFTER DELETE ON usuarios
FOR EACH ROW
BEGIN
  -- Eliminamos todos los permisos asociados al usuario que ya no existe
  DELETE FROM permisos
  WHERE id_usuario = OLD.id_usuario;
END;
//

-- Restauramos el delimitador estándar
DELIMITER ;
```

Descripción y uso

1. **Trigger permisos_after_insert**
   - Se ejecuta **después** de cada INSERT en usuarios.
   - Comprueba NEW.activo; si vale 1, inserta dos filas en permisos (Lectura y Escritura).
2. **Trigger permisos_after_delete**
   - Se ejecuta **después** de cada DELETE en usuarios.
   - Usa OLD.id_usuario para eliminar todos los permisos de ese usuario.

A partir de este momento, cada vez que hagas

```
INSERT INTO usuarios (id_usuario, nombre, activo) VALUES (123, 'Ana', 1);
```

se crearán automáticamente las dos filas correspondientes en permisos.

Y cuando hagas

```
DELETE FROM usuarios WHERE id_usuario = 123;
```

todos sus permisos se borrarán de la tabla permisos

10. Excepciones: Tipos y tratamiento

MySQL no maneja excepciones como algunos otros lenguajes de programación.

En su lugar, utiliza la señalización de errores y mensajes de error para indicar situaciones anormales que pueden ocurrir durante la ejecución de una consulta o un procedimiento almacenado.

10.1. Tipos

Las excepciones, que en MySQL son llamadas "errores", se clasifican en diferentes categorías según su naturaleza y origen.

A continuación, se detallan los tipos de excepciones más comunes en MySQL:

Errores de sintaxis	Ocurren cuando el código SQL enviado al servidor no sigue la sintaxis correcta. Por ejemplo, un error de sintaxis se produce cuando se escribe una consulta mal formada o se utiliza una palabra clave incorrectamente.
Errores de restricción	Ocurren cuando se viola una restricción de la base de datos, como una clave primaria, una clave foránea o una restricción UNIQUE. Por ejemplo, un error de restricción se produce cuando se intenta insertar una fila con un valor de clave primaria que ya existe en la tabla.
Errores de tiempo de ejecución	Ocurren durante la ejecución del código SQL y pueden estar relacionados con la falta de recursos o la interrupción del proceso de ejecución. Por ejemplo, un error de tiempo de ejecución se produce cuando se intenta ejecutar una consulta que tarda demasiado tiempo en completarse o se queda sin espacio en disco.
Errores de conexión	Ocurren cuando no se puede establecer una conexión con la base de datos debido a problemas de red, credenciales incorrectas o problemas con el servidor de la base de datos.

Errores del sistema	Ocurren cuando hay un problema con el servidor de la base de datos o el sistema operativo en el que se está ejecutando el servidor.

10.2. Tratamiento

Para manejar estas excepciones, MySQL ofrece la posibilidad de declarar manejadores de excepciones en los procedimientos almacenados y funciones utilizando las sentencias **DECLARE CONTINUE HANDLER FOR SQLEXCEPTION** o **DECLARE EXIT HANDLER FOR SQLEXCEPTION**.

Estas sentencias permiten especificar un código que se ejecutará cuando se produzca una excepción en el procedimiento almacenado o la función, siendo la principal diferencia entre ellas la forma en que afectan al flujo de ejecución del procedimiento almacenado o función:

- **DECLARE CONTINUE HANDLER** no termina la ejecución del procedimiento almacenado o función cuando se produce la excepción, simplemente captura la excepción y permite que la ejecución continúe. Por lo tanto, se puede utilizar para ignorar ciertas excepciones y continuar la ejecución del procedimiento almacenado o función.

- **DECLARE EXIT HANDLER** termina la ejecución del procedimiento almacenado o función cuando se produce la excepción. Esto significa que el control se transfiere inmediatamente al bloque END del procedimiento almacenado o función y la ejecución se detiene. Por lo tanto, se puede utilizar para abortar la ejecución del procedimiento almacenado o función cuando se produce una excepción crítica.

En ambos casos, la excepción capturada puede ser una excepción SQL o una señal personalizada.

Ejemplo de gestión de excepciones:

Para ilustrar la diferencia entre DECLARE CONTINUE HANDLER FOR SQLEXCEPTION y DECLARE EXIT HANDLER FOR SQLEXCEPTION, supongamos que tenemos la siguiente tabla llamada Usuarios:

```
CREATE TABLE Usuarios (
   id INT PRIMARY KEY,
```

```
  nombre VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL
);
```

Ahora, vamos a crear un procedimiento almacenado que intente insertar un nuevo usuario en la tabla Usuarios. Supongamos que queremos manejar la excepción que se produce cuando se intenta insertar un usuario con un correo electrónico que ya existe en la tabla.

Ejemplo con DECLARE CONTINUE HANDLER FOR SQLEXCEPTION:

```
DELIMITER //

CREATE PROCEDURE InsertarUsuario(IN id INT, IN nombre VARCHAR(50), IN email VARCHAR(100))
BEGIN
  DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
    SELECT 'Error: no se pudo insertar el usuario';

  INSERT INTO Usuarios (id, nombre, email) VALUES (id, nombre, email);
END
//

DELIMITER ;
```

En este ejemplo, hemos declarado un manejador de excepciones **DECLARE CONTINUE HANDLER FOR SQLEXCEPTION** que simplemente emite un mensaje de error cuando se produce una excepción. Ahora, vamos a llamar al procedimiento almacenado y a intentar insertar un usuario con un correo electrónico que ya existe:

```
CALL InsertarUsuario(1, 'Juan', 'juan@example.com');
```

Como el correo electrónico juan@example.com ya existe en la tabla Usuarios, se producirá una excepción.

Sin embargo, como hemos declarado un manejador de excepciones con DECLARE CONTINUE HANDLER FOR SQLEXCEPTION, la ejecución del procedimiento almacenado continuará y simplemente se mostrará el mensaje de error:

```
Error: no se pudo insertar el usuario
```

Ejemplo con DECLARE EXIT HANDLER FOR SQLEXCEPTION:

```
DELIMITER //

CREATE PROCEDURE InsertarUsuario(IN id INT, IN nombre VARCHAR(50), IN email VARCHAR(100))
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    SELECT 'Error: no se pudo insertar el usuario';

  INSERT INTO Usuarios (id, nombre, email) VALUES (id, nombre, email);
END//

DELIMITER ;
```

En este ejemplo, hemos declarado un manejador de excepciones **DECLARE EXIT HANDLER FOR SQLEXCEPTION** que emite un mensaje de error cuando se produce una excepción y termina la ejecución del procedimiento almacenado. Ahora, vamos a llamar al procedimiento almacenado y a intentar insertar un usuario con un correo electrónico que ya existe:

```
CALL InsertarUsuario(1, 'Juan', 'juan@example.com');
```

Como el correo electrónico juan@example.com ya existe en la tabla Usuarios, se producirá una excepción.

En este caso, como hemos declarado un manejador de excepciones con **DECLARE EXIT HANDLER FOR SQLEXCEPTION**, la ejecución del procedimiento almacenado se detendrá y se mostrará el mensaje de error:

```
Error: no se pudo insertar el usuario
```

Importante

La principal diferencia entre **DECLARE CONTINUE HANDLER FOR SQLEXCEPTION** y **DECLARE EXIT HANDLER FOR SQLEXCEPTION** es que el primero permite que la ejecución del procedimiento almacenado continúe después de una excepción, mientras que el segundo termina la ejecución del procedimiento almacenado inmediatamente después de una excepción.

11. Cursores

En MySQL, un cursor es un objeto que permite recorrer y manipular filas de una consulta SQL de manera individual.

Los cursores son especialmente útiles cuando se trabaja con conjuntos de datos grandes y se necesitan realizar operaciones en cada fila de manera individual.

Se utilizan en procedimientos almacenados y funciones, y se declaran utilizando la sentencia **DECLARE CURSOR**.

A continuación, se puede utilizar la sentencia **OPEN** para abrir el cursor, la sentencia **FETCH** para obtener las filas del cursor una por una, y la sentencia **CLOSE** para cerrar el cursor cuando se ha terminado de recorrer.

Ejemplo de cursor de lectura:

```
DELIMITER //

CREATE PROCEDURE listar_alumnos()
BEGIN
   -- Se definen las variables necesarias
   DECLARE done INT DEFAULT FALSE;
   DECLARE id INT;
   DECLARE nombre VARCHAR(255);
   DECLARE nota FLOAT;

   -- Se declara el cursor para recorrer la tabla alumnos
   DECLARE cur CURSOR FOR SELECT id, nombre, nota FROM alumnos;

   -- Se define el manejador de excepción para cuando no haya más filas en el
cursor
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;


   -- Se abre el cursor
   OPEN cur;


   -- Mientras no se haya llegado al final del cursor
   WHILE (done = FALSE) DO
```

```
    -- Se obtienen los valores de la siguiente fila del cursor
    FETCH cur INTO id, nombre, nota;

    -- Si se han obtenido valores
    IF NOT done THEN

      -- Se realizan las operaciones necesarias con los valores obtenidos
      SELECT CONCAT(nombre, ' tiene una nota de ', nota) AS datos_alumno;
    END IF;
  END WHILE;

  -- Se cierra el cursor
  CLOSE cur;
END
//
DELIMITER ;
```

Los comentarios explican cada parte del código, incluyendo la definición de variables, la creación del cursor, la definición del manejador de excepciones, la apertura del cursor, el ciclo WHILE para recorrer el cursor, la obtención de valores de cada fila con la sentencia FETCH, las operaciones con los valores obtenidos y la salida del ciclo cuando no hay más filas, y finalmente el cierre del cursor.

Ejemplo de cursor lectura y escritura:

En este ejemplo, volcaremos los datos de la tabla músicos en la tabla de copia de seguridad musicos_backup, que incluya la fecha y hora actual en un campo adicional llamado fecha_copia:

```
DELIMITER //
CREATE PROCEDURE backup_musicos_con_fecha()
BEGIN

  -- Se definen las variables necesarias
  DECLARE done INT DEFAULT FALSE;
  DECLARE nombre VARCHAR(255);
  DECLARE instrumento VARCHAR(255);
```

```
DECLARE fecha_copia DATETIME;

-- Se declara el cursor para recorrer la tabla musicos
DECLARE cur CURSOR FOR SELECT nombre, instrumento FROM musicos;

-- Se define el manejador de excepción para cuando no haya más filas en el cursor
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

-- Se abre el cursor
OPEN cur;

-- Se recorren las filas de la tabla musicos
WHILE (done = FALSE) DO

  -- Se obtienen los valores de la siguiente fila del cursor
  FETCH cur INTO nombre, instrumento;

  -- Si se han obtenido valores
  IF NOT done THEN

    -- Se asigna la fecha y hora actual
    SET fecha_copia = NOW();

    -- Se insertan los valores en la tabla de copia de seguridad
    INSERT INTO musicos_backup (nombre, instrumento, fecha_copia) VALUES (nombre, instrumento, fecha_copia);
  END IF;
END WHILE;
```

```
    -- Se cierra el cursor
    CLOSE cur;
END
//
DELIMITER ;
```

En este caso, hemos agregado una nueva variable llamada fecha_copia de tipo DATETIME.
Dentro del bucle WHILE, se asigna la fecha y hora actual a esta variable mediante la función NOW().
Después, se llama a la sentencia INSERT, incluyendo este nuevo campo en la tabla de copia de seguridad.
De esta manera, cada vez que se ejecute este procedimiento almacenado, se copiarán los datos de la tabla músicos en la tabla musicos_backup, junto con la fecha y hora en que se realizó la copia.

Ejemplo práctico: Gestión automatizada para el gimnasio FitLife

En este caso práctico veremos cómo, en la base de datos de FitLife, podemos combinar **eventos, disparadores, gestión de errores y cursores** para automatizar tareas de mantenimiento, garantizar integridad de datos y procesar lotes de información.

Para optimizar operaciones rutinarias, vamos a implementar:

1. **Evento** diario que consolida el total de nuevos socios por día.
2. **Disparador** que asigna automáticamente un plan de bienvenida cuando un socio se da de alta.
3. **Procedimiento con manejadores de excepción** para insertar pagos evitando duplicados.
4. **Procedimiento** que usa un **cursor** para generar un informe de asistencia semanal por clase.

Preguntas a resolver

1. ¿**Cómo schedulizar** el cálculo diario de nuevos socios para volcarlo en una tabla de estadísticas?
2. ¿**Cómo automatizar** el alta de un "plan de inicio" al insertarse un nuevo socio?

3. ¿**Cómo insertar un** pago capturando errores de clave duplicada sin abortar la sesión?
4. ¿**Cómo recorrer** todos los registros de asistencia de la última semana y generar un resumen por clase?

Solución

1. **Evento diario: resumen de altas**

1. Habilitar scheduler

```
SET GLOBAL event_scheduler = ON;


DELIMITER //
CREATE EVENT fitlife_resumen_altas
ON SCHEDULE EVERY 1 DAY
STARTS (CURRENT_DATE + INTERVAL 1 DAY)
DO
BEGIN
INSERT INTO estadisticas_altas (fecha, nuevas_altas)
SELECT CURDATE(), COUNT(*)
FROM socios
WHERE fecha_alta = CURDATE();
END//
DELIMITER ;
```

Activa el evento si no lo está

```
ALTER EVENT fitlife_resumen_altas ENABLE;
```

Qué hace: Cada día a medianoche inserta en estadisticas_altas el número de socios dados de alta ese mismo día.

2. **Disparador: plan de bienvenida**

```
DELIMITER //
CREATE TRIGGER trg_plan_bienvenida
```

```
AFTER INSERT ON socios
FOR EACH ROW
BEGIN
INSERT INTO planes_socio (socio_id, plan, fecha_inicio)
VALUES (NEW.id, 'Bienvenida FitLife', NEW.fecha_alta);
END//
DELIMITER ;
```

Qué hace: Tras crear un nuevo socio, asigna automáticamente el plan "Bienvenida FitLife" con la fecha de alta.

3. Procedimiento con manejo de excepciones: insertar pagos

```
DELIMITER //
CREATE PROCEDURE insertar_pago(
IN p_socio_id INT,
IN p_monto DECIMAL(8,2),
IN p_fecha DATE
)
BEGIN
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
BEGIN
Captura cualquier error (e.g. PK duplicada) y devuelve mensaje
SELECT 'Error: no se pudo registrar el pago' AS mensaje;
END;

INSERT INTO pagos (socio_id, monto, fecha)
VALUES (p_socio_id, p_monto, p_fecha);

SELECT 'Pago registrado correctamente' AS mensaje;
END//
DELIMITER ;
```

Qué hace: Intenta insertar un pago; si falla (por ejemplo, clave única), muestra un mensaje de error pero no aborta la sesión.

4. Procedimiento con cursor: informe semanal de asistencia

```
DELIMITER //
CREATE PROCEDURE informe_semanal_asistencia()
BEGIN
Variables de control
DECLARE done     INT DEFAULT FALSE;
DECLARE cls_id   INT;
DECLARE total    INT;

Cursor sobre clases con su total de asistencias en los últimos 7 días
DECLARE cur CURSOR FOR
SELECT c.id, COUNT(a.socio_id)
FROM clases c
LEFT JOIN asistencia a
ON a.clase_id = c.id
AND a.fecha BETWEEN CURDATE() - INTERVAL 7 DAY AND CURDATE()
GROUP BY c.id;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

Apertura
OPEN cur;
Recorre cada clase
REPEAT
FETCH cur INTO cls_id, total;
IF NOT done THEN
```

```
INSERT INTO reporte_asistencia_semanal (clase_id, total_asistencias, generado_el)
VALUES (cls_id, total, NOW());
END IF;
UNTIL done END REPEAT;
Cierre
CLOSE cur;
END//
DELIMITER ;
```

Qué hace: Cuenta la asistencia de cada clase en la última semana y vuelca esos totales en reporte_asistencia_semanal.

Con estas cuatro piezas —**evento, trigger, manejador de excepciones y cursor**— FitLife puede automatizar consolidaciones diarias, garantizar reglas de negocio en altas de socios, gestionar pagos con robustez y generar informes periódicos sin intervención manual.

12. Resumen

MySQL es un sistema de gestión de bases de datos relacionales que utiliza una sintaxis propia basada en el lenguaje SQL. Incluye una amplia variedad de palabras reservadas que se utilizan para realizar diferentes operaciones, como SELECT, INSERT, UPDATE, DELETE, entre otras. Además, MySQL permite utilizar variables de sistema y variables de usuario para almacenar información temporal. También proporciona varios tipos de datos compuestos, como arrays, estructuras y tablas, para gestionar datos más complejos.

Las funciones en MySQL permiten realizar operaciones matemáticas, de cadena de texto, de fecha y hora, y otras operaciones. Las estructuras de control de flujo, como los condicionales y bucles, permiten controlar el flujo de ejecución en las consultas y procedimientos almacenados.

Los procedimientos almacenados y subrutinas son bloques de código que se pueden ejecutar de forma repetitiva en MySQL. Las variables tienen un ámbito limitado dentro de cada procedimiento o subrutina.

MySQL también incluye eventos y disparadores que se pueden utilizar para automatizar tareas, como la ejecución de consultas periódicas o la gestión de cambios en las tablas. Además, proporciona un manejo de excepciones para capturar y tratar errores durante la ejecución de consultas.

Finalmente, MySQL permite el uso de cursores para recorrer y manipular conjuntos de resultados de forma programática.

Glosario

- **Palabras reservadas**: son palabras que tienen un significado específico en MySQL y se utilizan para realizar diferentes operaciones, como SELECT, INSERT, UPDATE, DELETE, entre otras.

- **Comentarios**: son textos que se pueden agregar en el código SQL para explicar su función y facilitar su comprensión. En MySQL se pueden utilizar dos tipos de comentarios: los de una sola línea (precedidos por "--") y los de varias líneas (encerrados entre "/" y "/").

- **Variables del sistema**: son variables predefinidas en MySQL que se utilizan para almacenar información relacionada con la configuración y el funcionamiento del sistema, como la fecha y hora actual o el nombre del usuario actual.

- **Variables de usuario**: son variables que se definen por el usuario y se utilizan para almacenar información temporal en la base de datos, como resultados intermedios de una consulta o datos que se utilizan en un procedimiento almacenado.

- **Tipos de datos compuestos**: son tipos de datos que permiten almacenar estructuras de datos más complejas que un simple valor, como arrays, estructuras y tablas.

- **Funciones**: son bloques de código predefinidos en MySQL que se utilizan para realizar operaciones matemáticas, de cadena de texto, de fecha y hora, y otras operaciones.

- **Estructuras de control de flujo**: son bloques de código que permiten controlar el flujo de ejecución de una consulta o procedimiento almacenado en función de ciertas condiciones. Ejemplos de estructuras de control de flujo incluyen los condicionales (IF, CASE) y los bucles (WHILE, FOR).

- **Procedimientos almacenados**: son bloques de código que se pueden almacenar en la base de datos y ejecutar de forma repetitiva para realizar operaciones específicas en la base de datos.

- **Subrutinas**: son bloques de código similares a los procedimientos almacenados, pero que no se almacenan en la base de datos y se ejecutan dentro del contexto de otro procedimiento o consulta.
- **Ámbito de variables**: se refiere a la parte del código donde una variable puede ser accesible. En MySQL, el ámbito de una variable se limita al procedimiento o subrutina donde se declara.

- **Eventos y disparadores**: son características de MySQL que se utilizan para automatizar tareas, como la ejecución de consultas periódicas o la gestión de cambios en las tablas.

- **Excepciones**: son errores que se producen durante la ejecución de una consulta o procedimiento almacenado. En MySQL, se pueden utilizar manejadores de excepciones para capturar y tratar estos errores.

- **Cursores**: son objetos que permiten recorrer y manipular conjuntos de resultados de forma programática. En MySQL, se pueden utilizar cursores para realizar operaciones más complejas en las consultas y procedimientos almacenados.

Tech Hub <FP> euroformac

Bibliografía

MySQL. (2014). MySQL Documentation. Retrieved from https://dev.mysql.com/doc/

Vázquez, A. (2021). MySQL: Lenguaje y diseño de bases de datos. Madrid: Ra-Ma Editorial.

Harrison, G. (2006). MySQL Stored Procedure Programming. O'Reilly Media.

Tech Hub <FP> euroformac